



School of Physics, Engineering and Computer Science
MSc Computer Science
Artificial Intelligence and Robotics

Module: MSc CS Project

Final Project Report

FPGA-Based Driver Distraction Detection System Using Image Classification



Name : Dulhara Anjana Langappuli
Student ID : 22044421
Level : Level 7
Academic Year : 2025-26
Supervisor : Professor Saeid Gorgin

I confirm that this report has been critically proof-read and quality checked by me, and that it is free from grammar, spelling and formatting errors to the best of my knowledge, meeting the standards of clarity, coherence and presentation expected of this submission.

Declaration

This report is submitted in partial fulfilment of the requirement for the degree of Master of Science in Computer Science (Artificial Intelligence and Robotics) at the University of Hertfordshire.

I hereby declare that the work presented in this project and report is entirely my own, except where explicitly stated otherwise. All sources of information and ideas, whether quoted directly or paraphrased, have been properly referenced in accordance with academic standards. I understand that any failure to properly acknowledge the work of others could constitute plagiarism and may result in academic penalties.

I did not use human participants in my MSc Project.

I hereby **give** permission for the report to be made available on the university website provided the source is acknowledged.

Signature:



Name: Dulhara Anjana Langappuli

Student ID: 22044421

Date: 30th August 2026

Contents

List of Abbreviations	viii
Abstract	1
1 Introduction	2
1.1 Problem overview	2
1.2 Current issues and the gap this project addresses	2
1.3 Project details	3
1.4 Aims and objectives	3
1.5 Research questions	3
1.6 Feasibility, commercial context and risk	4
1.7 Report structure	4
2 Literature Review	5
2.1 How the literature for this project was selected	5
2.2 Image-based driver distraction detection	7
2.3 Subject-wise evaluation, dataset bias and domain shift	7
2.4 Multi-view and cross-view driver monitoring	7
2.5 Low-light, infrared and near-infrared driver monitoring	8
2.6 Quantisation and embedded/FPGA deployment	9
2.7 Gaps identified and positioning of this project	9
3 Methodology	11
3.1 Project management approach	11
3.2 Dataset preparation and splitting strategy	11
3.2.1 Single-view baseline: State Farm	11
3.2.2 Multi-view extension: SAM-DD	11
3.2.3 Lighting extension: the Low-Light Driver Distraction Dataset	12
3.2.4 Subject-wise integrity across every dataset and stage	13
3.3 Model selection and training	13
3.4 Quantisation and on-device deployment	14
3.4.1 Model conversion	14
3.4.2 Board bring-up and the ARM32 runtime issue	14
3.5 FPGA alert-controller design and verification	15
3.6 Alert delivery, tools and test strategy	15
3.7 Ethical, legal, professional and social issues	16

4	Quality and Results	17
4.1	Single-view baseline: model comparison (RQ1)	17
4.2	Cross-view generalisation (RQ2)	19
4.3	Lighting-domain robustness (RQ3)	22
4.3.1	The day-only baseline collapses	22
4.3.2	Training on real night data recovers most of the gap	23
4.4	Quantisation: accuracy, size and latency (RQ4)	27
4.4.1	Model size	27
4.4.2	Accuracy	27
4.4.3	The consolidated software-to-hardware comparison	27
4.5	On-device deployment on the ZC702 (RQ4)	28
4.5.1	Establishing runtime correctness first	28
4.5.2	Accuracy on real hardware	28
4.6	FPGA alert-controller verification and synthesis (RQ5)	30
4.7	Class-level error analysis across all three stages	31
4.8	Summary against the research questions	32
5	Evaluation and Conclusion	33
5.1	Final evaluation against objectives	33
5.2	Project management	33
5.3	Insights gained	34
5.4	Reflection on challenges	34
5.5	Comparison to the literature	35
5.6	Limitations	35
5.7	Commercial context and future work	35
5.8	Conclusion	36
	References	37
A	Systematic Search and Screening Detail	40
B	Dataset Splits and Composition	41
C	Full Per-Class Classification Reports	43
D	Quantisation Raw Figures	45
E	FPGA Synthesis Report Excerpts	46
F	Alert Delivery Path and Its Verification	48
G	Physical Buzzer Alert Output	51
G.1	The board could not control the header pin at all	51
G.2	The overlay	51
G.3	Root permission and two unrelated Jupyter defects	53
G.4	Driving a passive buzzer	53
G.5	Integration	53

H Selected Code Excerpts	54
I Repository and Notebook Map	56

List of Figures

2.1	PRISMA-style selection process for the systematic search underpinning this review (90 records identified, 20 core studies selected).	5
2.2	Reported accuracy across the core studies. Differences reflect the datasets and evaluation protocols as well as model performance.	6
2.3	Deployment maturity evidence across the selected studies. Measurements collected on embedded target devices are less common than offline evaluations.	6
2.4	The surveyed research landscape and this project’s pathway through it, produced during the systematic survey that preceded the practical work.	10
2.5	Dataset and method coverage across the core studies. Sparse regions indicate combinations that are under-explored rather than combinations that do not work.	10
3.1	Class distribution across the State Farm subject-wise train/validation/test splits.	12
3.2	Confirming directly on the board that this processor is missing the math instruction the published software assumed it would have.	14
4.1	Test accuracy of the custom CNN, MobileNetV2 and EfficientNet-B0 on the State Farm subject-wise test set.	18
4.2	Macro and weighted F1 scores of the three single-view baseline models.	18
4.3	Accuracy against laptop throughput for the three baseline models, showing the trade-off underlying the MobileNetV2 selection.	19
4.4	Cross-view MobileNetV2 training and validation accuracy across the frozen-backbone and fine-tuning phases.	20
4.5	Cross-view MobileNetV2 training and validation loss across the frozen-backbone and fine-tuning phases.	20
4.6	Row-normalised confusion matrix for the frozen-backbone cross-view model on the held-out mixed-view test set.	21
4.7	Row-normalised confusion matrix for the fine-tuned cross-view model on the held-out mixed-view test set.	21
4.8	Qualitative predictions from the fine-tuned cross-view model on held-out test images spanning both camera views.	22
4.9	Low-light MobileNetV2 training and validation accuracy across the frozen-backbone and fine-tuning phases.	24
4.10	Low-light MobileNetV2 training and validation loss across the frozen-backbone and fine-tuning phases.	24

4.11	Row-normalised confusion matrix for the day-trained MobileNetV2 evaluated directly on the 1,500 held-out real night images. The predictions collapse mainly into two classes.	25
4.12	Row-normalised confusion matrix for the frozen-backbone low-light model on the held-out mixed day-and-night test set.	25
4.13	Row-normalised confusion matrix for the fine-tuned low-light model on the held-out mixed day-and-night test set.	26
4.14	Qualitative predictions from the fine-tuned low-light model on held-out real night-time images.	26
4.15	On-device (ZC702, TFLite INT8) against reference (Keras float32) accuracy on the same 300-image held-out sample, before and after fine-tuning.	29
4.16	A real inference run on the ZC702, showing the predicted class and confidence reported in Section 4.5.2.	29
4.17	Post-synthesis resource utilisation for the alert controller on xc7a35tcpg236-1. .	30
5.1	Detailed project schedule, with the literature survey and report writing shown as parallel tracks.	34
B.1	State Farm dataset composition.	42
E.1	Vivado’s own utilization report view for the synthesized alert controller, confirming the 18 LUTs / 13 registers reported in Table E.1.	46
E.2	Vivado’s timing summary panel, confirming the worst-negative-slack figure reported in Table E.2.	47
F.1	The self-checking testbench run in Vivado’s simulator: the waveform shows alert correctly staying low, then rising after sustained distraction and clearing after sustained safe driving, and the Tcl Console confirms all ten checks passed, matching Table F.1 (Layer 1).	49
F.2	The FastAPI backend receiving a real alert (top) and the same record persisted in its SQLite database (bottom), confirming the HTTP 200 and stored-record result reported in Table F.1.	49
F.3	A real alert received on the Android client, showing the triggering frame and timestamp posted by the board.	50
G.1	The buzzer overlay’s Vivado block design. A Zynq7 Processing System block provides AXI/clock/reset plumbing to the already-running processor. A single-bit AXI GPIO block is the only new circuit, exposed externally as buzzer_o on package pin V7.	52
G.2	The ZC702 board with the passive buzzer wired directly to header pin J62 (PMOD2_0), driven by the GPIO overlay described in this appendix.	52
I.1	The project’s GitHub repository.	56

List of Tables

3.1	Subject-wise splitting applied consistently across all datasets and stages. No subject appears in more than one partition and all reported test figures come from the held-out subjects listed here. The Low-Light daytime rows are excluded from training for the reason given in Section 3.2.3.	13
4.1	Single-view model comparison on the State Farm subject-wise test set (3,086 images, 4 held-out drivers).	17
4.2	Deployment-oriented measurements on the development laptop (GPU-accelerated). These are laptop figures and are not comparable with the on-device figures in Table 4.6.	17
4.3	Cross-view MobileNetV2 accuracy, frozen-backbone against fine-tuned, on the 17,992-image held-out mixed-view test set.	19
4.4	Lighting-domain robustness. The baseline column is the day-trained model applied to night images with no retraining. All test figures come from held-out drivers. .	23
4.5	Quantisation accuracy cost, measured on different held-out samples. The two loss figures differ because INT8 quantisation error is data-dependent, not because either measurement is unreliable.	27
4.6	Consolidated software, deployment and hardware comparison for the fine-tuned cross-view MobileNetV2 and the FPGA alert controller. Laptop and on-device figures are reported separately and are not interchangeable.	28
4.7	Post-synthesis results for the alert controller, <code>xc7a35tcpg236-1</code> , out-of-context. Raw report excerpts are in Appendix E.	30
4.8	Selected per-class results across stages (P = precision, R = recall). Full tables in Appendix C.	31
A.1	Search sources and initial records identified (before duplicate removal).	40
B.1	State Farm subject-wise split, by driver identifier.	41
B.2	Composition of the unified multi-view metadata table (170,316 rows).	41
C.1	Per-class F1 for the three single-view baseline models, State Farm test set.	43
C.2	MobileNetV2 full per-class report, State Farm subject-wise test set (accuracy 84.67%).	43
C.3	Cross-view MobileNetV2, frozen-backbone against fine-tuned, mixed-view test set (17,992 images).	44

C.4	Lighting stage: day-only baseline (night-only test set, 1,500 images), frozen-backbone and fine-tuned (merged day+night test set, 4,586 images). The baseline's zero-recall rows are the collapse described in Section 4.3.	44
D.1	Exact model artefact sizes.	45
D.2	Quantisation before/after measurements. All samples are drawn from held-out subject-wise test partitions.	45
E.1	Slice logic utilisation (from <code>utilization.rpt</code>).	46
E.2	Design timing summary, clock constrained at 100 MHz.	47
F.1	Three-layer verification of the alert-delivery path.	48

List of Abbreviations

Abbreviation	Meaning
AI	Artificial Intelligence
ARM	Processor architecture used by the ZC702's Cortex-A9 cores
AXI	Advanced eXtensible Interface
BRAM	Block Random-Access Memory
CNN	Convolutional Neural Network
CPU	Central Processing Unit
DSP	Digital Signal Processing
F1	Harmonic mean of precision and recall
FPGA	Field-Programmable Gate Array
FPS	Frames per second
GPIO	General-Purpose Input/Output
INT8	Eight-bit integer representation used for model quantisation
IR	Infrared
LUT	Lookup Table
NIR	Near-Infrared
NoIR	Camera without an infrared-blocking filter
PRISMA	Preferred Reporting Items for Systematic Reviews and Meta-Analyses
RGB	Red, Green and Blue colour representation
RTL	Register-Transfer Level
SoC	System-on-Chip
TFLite	TensorFlow Lite
UART	Universal Asynchronous Receiver-Transmitter
USB	Universal Serial Bus
VFP	Vector Floating Point
WNS	Worst Negative Slack
XNNPACK	Optimised neural-network inference library used by TensorFlow Lite

Abstract

Most published driver-distraction detection systems are trained and evaluated using images from one fixed camera position and one lighting condition, while deployment speed is often reported only on a host computer. This project investigates how performance changes when camera viewpoint and lighting vary and whether a complete detection-and-alert pipeline can be demonstrated on low-cost embedded hardware.

A custom CNN, MobileNetV2 and EfficientNet-B0 were trained on the State Farm Distracted Driver Detection dataset using a subject-wise split, which kept test drivers out of the training data. MobileNetV2 performed best, reaching 84.67% test accuracy and 84.01% macro F1. Camera-view robustness was then evaluated using State Farm and the multi-view SAM-DD dataset. A frozen-backbone model achieved 75.94% mixed-view accuracy, with uneven results between views. Fine-tuning increased this to 91.76%. For lighting, the daytime model was tested on real infrared-illuminated night footage and achieved only 19.53% accuracy, with zero recall in eight of the ten classes. Training with real night data raised night-time accuracy to 71.13%, although daytime accuracy fell by 3.5 percentage points.

The fine-tuned model was converted to INT8 TensorFlow Lite, reducing its size from 20.78 MB to 2.60 MB at a cost of 2.7 accuracy points. It was deployed on a Xilinx ZC702 after an instruction-set incompatibility in the available runtime was diagnosed and corrected. The board achieved 77.3% accuracy on the deployment sample, while a separate correctness test produced agreement on 100 out of 100 predictions against the reference runtime. However, inference took approximately 9.15 seconds per image, making latency the main deployment limitation. The alert-decision logic was also verified using real model outputs and synthesised for a low-cost FPGA target using 18 lookup tables and 13 flip-flops, with 7.276 ns of positive timing slack.

The findings show that robustness to viewpoint and lighting must be measured and trained explicitly. They also demonstrate that low-cost embedded deployment is feasible, although the present CPU implementation is not fast enough for a practical real-time system.

Chapter 1

Introduction

1.1 Problem overview

Distracted driving is a major road-safety problem. Activities such as using a phone, eating, drinking, adjusting vehicle controls or talking to a passenger can take a driver's attention away from the road. Camera-based classification offers a practical way to recognise these behaviours and generate an alert when sustained distraction is detected.

The difficulty is that many published systems are developed and evaluated under controlled conditions, usually with one camera position and one lighting condition. Strong accuracy under those conditions does not guarantee comparable performance from another viewpoint or at night. A usable in-car system must also run on suitable hardware and turn individual frame predictions into stable alerts. This project examines these modelling and deployment problems as parts of the same system.

1.2 Current issues and the gap this project addresses

Three gaps motivate this project. Chapter 2 assesses each gap against the existing literature.

First, camera-view robustness is rarely tested directly. In data where multi-view options exist, much previous research either uses a single view or measures performance inside a viewpoint rather than across viewpoints. The existence of left and right distraction classes further complicates this problem because a frontal camera provides a mirrored view in relation to the side camera view and thus "texting left" performed with the former will look like "texting right" when captured with the latter. Pooling across views without addressing this discrepancy runs the risk of quietly skewing labels.

Second, while lighting robustness is a widely recognized problem, its evaluation is rarely conducted with actual data. Where low-lighting scenarios are included in evaluations, researchers frequently use artificially dimmed daytime images or infrared-like coloring, which is a perfectly acceptable proxy for robustness but is no replacement for real low-light images. Real low-light data captures effects that cannot be created synthetically, such as color attenuation, the impact of near infrared illumination on skin and fabric reflections and camera noise. This project initially considered doing a synthetic experiment of this sort but abandoned it in favor of real low-light data once it became available.

Third, the progression from a trained model to a functioning alert system is generally seen as a promising future development rather than something to be actually implemented. Frames

per second measurements are usually obtained using development workstation scale benchmarks rather than on realistic hardware configurations. Quantization effects on accuracy and model size are rarely considered and the process used to transform frame-wise predictions into an alert is even more rarely explained and validated empirically.

1.3 Project details

The project is built in two connected phases that match these gaps.

Phase 1 - AI robustness. A single-view baseline is set up first. Custom CNN, MobileNetV2 and EfficientNet-B0 are trained on the State Farm Distracted Driver Detection dataset (10 classes, 22,424 images) using a subject-wise split. These three models are then tested and compared in terms of accuracy, macro F1, per class recall and throughput. Next, two robustness tests are run on real data. First, camera-view robustness test consists of combining State Farm (side view) and SAM-DD dataset, which contains images in pairs from front- and side-views. Second, lighting robustness is tested by combining State Farm and nighttime subset of the Low-Light Driver Distraction Dataset (Saad et al., 2020), recorded in active infrared light. This test evaluates the drop in accuracy of day-only model before any intervention is taken.

Phase 2 - hardware. The fine-tuned model is converted into quantized INT8 TensorFlow Lite format and deployed on the Xilinx ZC702 board (Zynq-7000) for the reason that this board contains a general-purpose CPU ARM Cortex-A9 core that can perform a full CNN, unlike an FPGA board without a core on board. Real latency and accuracy is measured instead of being estimated. Decision stage with hysteresis logic in order to prevent flickering alerts is implemented in Verilog, verified through simulation by comparing the decision stage predictions to the predictions made by the trained model. Finally, the alert pathway from the board to a backend and Android client is set up.

1.4 Aims and objectives

1. To prepare and use suitable driver-distraction image data, including real multi-view and real low-light data, for training and testing a camera-view-robust and lighting-robust classification system.
2. To build and test a prototype running on real, low-cost embedded hardware, with an alert output that reacts to sustained distraction rather than a single momentary prediction and to measure the accuracy, size and latency cost of getting there.

1.5 Research questions

- RQ1** How do a custom CNN, MobileNetV2 and EfficientNet-B0 compare when the test drivers are not present in the training data?
- RQ2** How much does accuracy change when a model is tested on a different, unseen camera view and does fine-tuning on multi-view data close that gap?
- RQ3** How much accuracy does a day-trained model lose on real infrared-illuminated footage and how much can be recovered by training on real night data rather than synthetic approximations?

- RQ4** What does INT8 quantisation and embedded deployment cost in accuracy, model size and latency before and after conversion and how do laptop figures differ from real on-device performance?
- RQ5** Which part of a complete pipeline, the classifier or the decision logic after it, can actually be built and checked in FPGA hardware in the time available?

Every experiment in Chapter 4 is tied to one of these questions. Section 4.8 gives the answer each one produced.

1.6 Feasibility, commercial context and risk

The scope has been defined to include aspects which could be physically implemented and measured, as opposed to purely simulated. A fully functional CNN accelerator based on an FPGA was seen to be too challenging for the timeline and was thus discarded. The ZC702's Cortex-A9 core runs the classifier and the fabric is used for the decision making stage. This solution was known in advance to be a fall-back plan from the very start from the Interim Progress Report.

The business case of the project is based on cost, rather than novelty. As driver monitoring becomes mandatory for new cars of certain categories in several countries, current systems are developed by the manufacturers. However, in the retrofit and fleet sector, which is the target market, older cars, taxis, delivery vans, driving schools, et cetera are equipped with hardware of much more modest budget. That is why the project aims at using components costing in tens of pounds, as opposed to thousands. Among other risks are the possibility of drivers not taking kindly to surveillance, legal issues arising due to false alerts and the obvious gap between an experimental proof of concept and a commercially viable product, which is covered in more detail in Chapter 5.

The key technical problem that emerged during the project was an incompatibility at the low level of the single existing TensorFlow Lite runtime for this board and the actual instruction set (Section 3.4.2). Fixing this problem instead of giving up on local inference was one of the main technical contributions of the project.

1.7 Report structure

Chapter 2 reviews prior work related to distraction detection, cross view and lighting robustness and embedded/FPGA-based deployment. Chapter 3 covers data preparation, model training, quantisation, FPGA implementation, buzzer-overlay implementation and testing strategy. Chapter 4 contains the results with critical analysis, including a software/hardware performance comparison. Chapter 5 evaluates the project according to its goals, literature review, project management, limitations and future work.

Chapter 2

Literature Review

2.1 How the literature for this project was selected

This review was based on a PRISMA-style search and screening exercise completed in June 2026 (Page et al., 2021; PRISMA, 2020). The search covered IEEE Xplore, Google Scholar, SpringerLink, MDPI, Elsevier, ScienceDirect, arXiv, dataset records and citation tracing. Search terms were organised around three parts of the project which are RGB driver-distraction classification, embedded deployment and lighting robustness.

The search identified 90 records, of which 22 were duplicates. Following title, abstract and full-text screening, 31 studies were retained: 20 core studies and 11 supporting studies. Figure 2.1 summarises the process, while the complete search and exclusion details are provided in Appendix A.

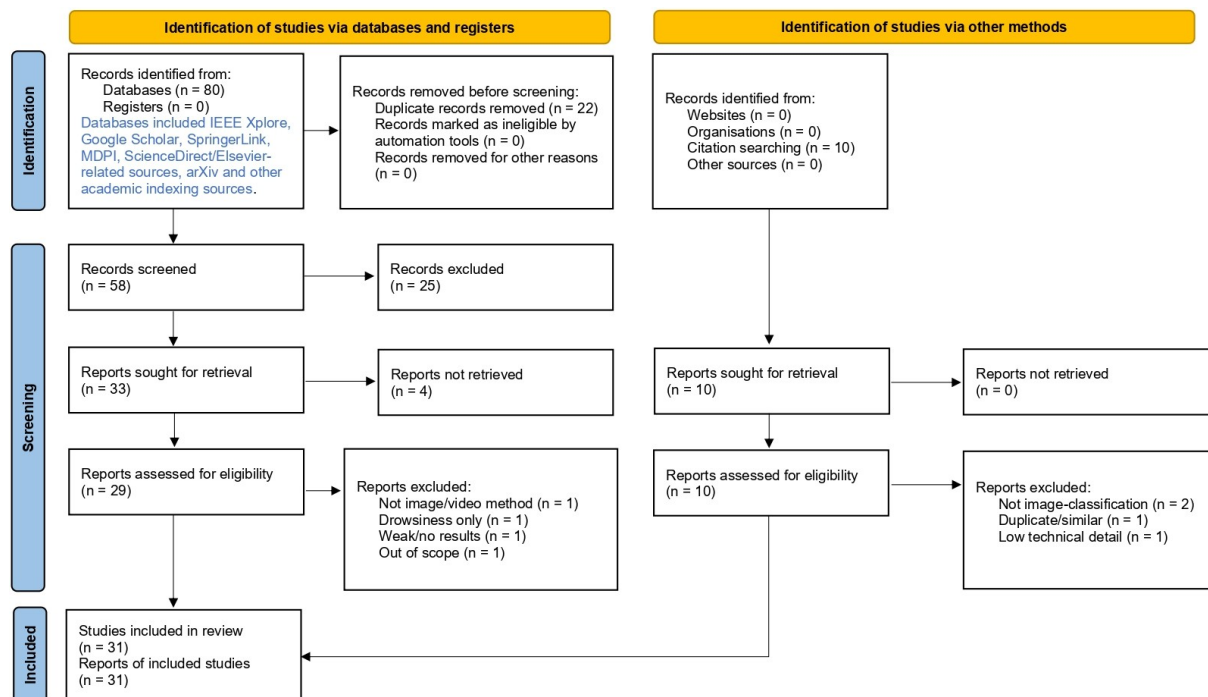


Figure 2.1: PRISMA-style selection process for the systematic search underpinning this review (90 records identified, 20 core studies selected).

Two patterns from the review shaped this project. First, accuracy values from different studies

are not directly comparable when their datasets and splitting protocols differ. For example, a high result from a random image-wise split may provide weaker evidence of generalisation than a lower result from a subject-wise split. Second, claims about embedded deployment are more common than measurements collected on the target device. These findings motivated the consistent use of subject-wise evaluation and the separation of laptop and on-device measurements in Chapter 4.

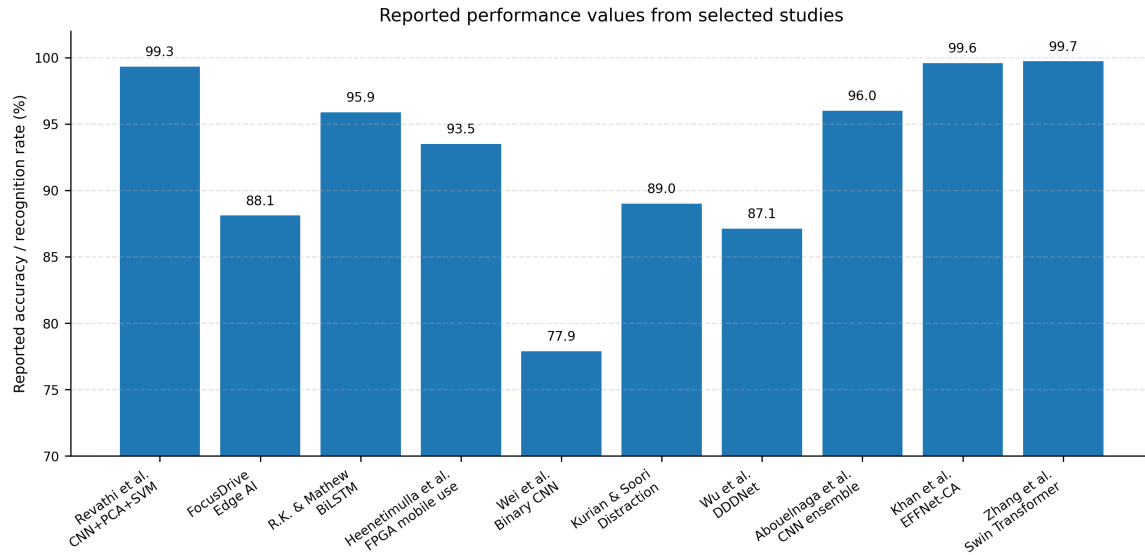


Figure 2.2: Reported accuracy across the core studies. Differences reflect the datasets and evaluation protocols as well as model performance.

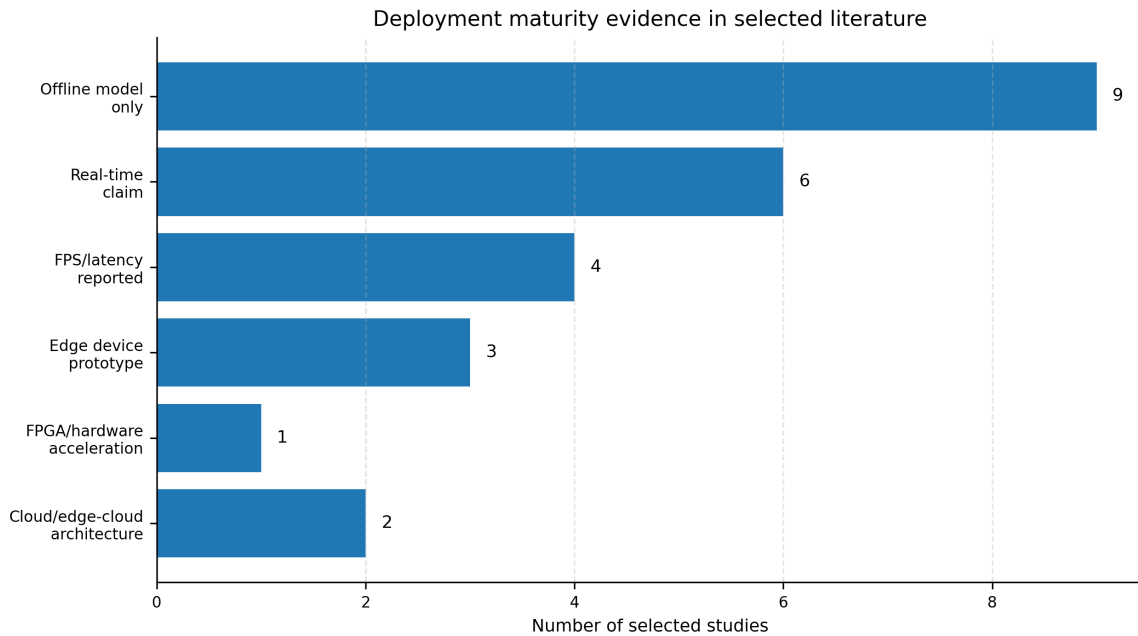


Figure 2.3: Deployment maturity evidence across the selected studies. Measurements collected on embedded target devices are less common than offline evaluations.

2.2 Image-based driver distraction detection

The application of CNNs for image-based distraction detection is a well-established topic. In particular, Valeriano et al. (2018) train a CNN on the State Farm dataset used in this research, demonstrating good within-view results and showing that ten classes taxonomy can be learned using only one side-mounted camera without any other sensors. AlShalfan and Zakariah (2021) and Aljasim and Kashef (2022) continue research in this direction. While the first paper evaluates several deep architectures on the same task, the second paper presents an ensemble technique that, once the distraction is detected, gives a recommendation as to how to respond to it, thus considering classification a part of the bigger decision-making process – an approach close to the one taken in this paper, where the decision stage after the classifier is carefully designed.

Abouelnaga et al. (2018) and Eraqi et al. (2019) have introduced AUC Distracted Driver dataset and an ensemble method using it, thus providing evidence that the State Farm taxonomy is transferable to an independently collected dataset. It has become the motivation behind the choice of AUC V2 during the multi-view stage, despite being unavailable at the time of this project. General reviews by Kashevnik et al. (2021), Horgan et al. (2021) and Dontoh et al. (2025) confirm the existence of the above pattern rather than provide just one example: the reported systems are mostly evaluated on a single camera position and under a certain lighting condition, which are seen as properties of the dataset rather than its variables to be tested.

MobileNetV2 and EfficientNet-B0 were selected as transfer-learning candidates because they offer different compromises between accuracy and computational cost. MobileNetV2 was designed for resource-constrained deployment (Sandler et al., 2018), while EfficientNet-B0 provides a compact benchmark based on balanced scaling of network depth, width and resolution (Tan and Le, 2019).

2.3 Subject-wise evaluation, dataset bias and domain shift

The issue that repeatedly emerges in the context of this field and is handled in this work rigorously is subject leakage. If a driver appears in the training and test sets simultaneously, the split into training and testing samples randomly based on images overestimates the real-world accuracy of a model, as it is able to use the appearance features (clothing, body build, seat position, etc.) of the driver rather than his behavior and consecutive video frames make this effect even more pronounced. This becomes the main reason why numbers presented in this field cannot be easily compared between papers. Thus, the subject-wise protocol described in Section 3.2 is consistently used on all datasets and project stages. Wang and Wu (2023) address a different but similar problem of domain shift due to changes in the conditions of recordings and demonstrate a significant drop of performance in such case. This result is consistent with the ones presented here, although the magnitude of drop under the lighting shift in real conditions (Section 4.3) is significantly higher.

2.4 Multi-view and cross-view driver monitoring

The prevalence of evaluations based on single-view data is not due to the lack of multi-view data and techniques. Jegham et al. (2019) created the MDAD, a multi-view, multi-modal driver action dataset built off synchronized side and front cameras, akin to SAM-DD (Yang et al., 2024)

used in the current project, proving that synchronization-based generation of cross-view data is a known approach, not a specific feature of this work. Ortega et al. (2020) and Wang et al. (2023) extend the approach to larger multi-modal datasets. A different approach is taken by Sengar et al. (2023), who extract the pose data from each viewpoint and combine it with a vision transformer, thus showing that viewpoint-aware features may be a plausible alternative to the fine-tuning approach employed in this work.

Most relevantly, Bhalla et al. (2025) tackle the cross-view issue in a two-step process: they learn viewpoint-invariant features through contrastive learning across viewpoints and then adapt these features to different sensors without target labeling, obtaining significant improvements over the older baselines. This approach is far superior to the technique used in this work of fine-tuning of the last 30 layers of an already trained backbone network using pooled multi-view data. However, the comparison is telling because despite the relatively simple fine-tuning technique used in this work, it was still able to recover most of the accuracy losses from the evaluation of the frozen-backbone network in a cross-view setup (Section 4.2). Therefore, fine-tuning of the model on real paired data is a reasonable middle-ground for a project of this type, although contrastive adaptation is a more principled approach.

One distinctive thing about this project compared to the surveyed literature is that it explicitly evaluates the cross-view gap in a frozen backbone "before" taking any steps to address it. Very few of the surveyed papers provide an equivalent "before" baseline, making it hard to judge the typical size of the raw gap for a MobileNetV2-class model and the degree to which the claimed improvements are due to the suggested method vs just having multi-view training data available.

2.5 Low-light, infrared and near-infrared driver monitoring

Nighttime driver monitoring could be considered the most relevant application scenario for a driver monitoring system of this kind, yet it appears to be underrepresented in the RGB-based classification literature. The most directly relevant work is Saad et al. (2020), who created the Novel Driver Distraction Dataset with low-light support (Hussien et al., 2020), specifically because the benchmark datasets of the time lacked nighttime data and this is the dataset used in the current project (Section 3.2.3). Janveja et al. (2020) position low-light driver state monitoring as an embedded systems problem rather than a modeling one.

Methodologically, the key work in this area is Shen and Lin (2023). It proposes a way to solve both day and nighttime driver state classification using visible-to-infrared image translation, producing images looking like infrared ones. This paper exemplifies the strengths and weaknesses of the synthetic approach: it offers a way to augment a small pool of night training data but is explicitly presented as an approximation, not a substitute for the real deal. This particular distinction is crucial in the current project's context. The initial idea for the lighting stage of this project included the translation technique described in Shen and Lin (2023). However, the intentional decision was made to abandon it once real low-light data was available, since a synthetic transformation cannot duplicate the effect that infrared illumination has on a scene, namely, the loss of color information, the way skin and fabric reflect near-infrared light and the noise in the image caused by the low-light operation of the camera. That is why Section 4.3 relies on the real footage with infrared illumination and all conclusions about lighting robustness made in this report are about real footage, not an approximation.

2.6 Quantisation and embedded/FPGA deployment

Edge deployment introduces costs that model accuracy alone does not show. Abushahla et al. (2025) identify INT8 post-training quantisation as a common first step for reducing the storage and computational cost of edge-AI models. The accuracy reductions measured in this project, ranging from 0.8 to 4.6 percentage points across different samples, are consistent with the small but data-dependent losses discussed in that literature. Hariharan et al. (2023) similarly examine the practical requirements of deploying AI-based driver-monitoring systems.

The FPGA literature shows the difference between running a model on an embedded processor and accelerating it in programmable logic. Parra et al. (2023) describe a custom AXI-Lite accelerator for a TensorFlow-Lite-quantised CNN on a Zynq device. More directly, Lu et al. (2025) implement a MobileNetV2-based driver-behaviour accelerator on the same XC7Z020 device family used here, achieving 13.05 FPS at 1.25 W.

Those systems are substantially more complex than the implementation in this project. Here, the classifier runs on the Cortex-A9 processor and the FPGA fabric handles only the alert-decision stage. This choice kept the work achievable within the project scope, but produced a much lower speed of approximately 9.15 seconds per inference. Reporting that limitation alongside the successful deployment gives a more realistic comparison with dedicated accelerators.

2.7 Gaps identified and positioning of this project

Figure 2.4 demonstrates how this project fits into the landscape of the surveyed literature, while Figure 2.5 shows the regions of applicability of different methods families to different datasets. The regions without the data (areas without information in comparison to areas with information) are equally valuable. There are four gaps revealed, each answering one of the research questions of Section 1.5.

1. **Camera-view robustness is claimed more often than it is measured.** Few papers report the size of the problem for a simple baseline before fixing it (RQ2).
2. **Lighting robustness is usually approximated rather than tested on real infrared-illuminated data** (Shen and Lin, 2023). This project measures the day-to-night collapse on real captured footage (RQ3).
3. **Quantisation and deployment costs are often reported partially or not at all.** Laptop throughput is often given with no matching on-target measurement (RQ4).
4. **The decision stage that turns per-frame predictions into a stable alert is rarely treated as its own designed and tested part of the system** (RQ5).

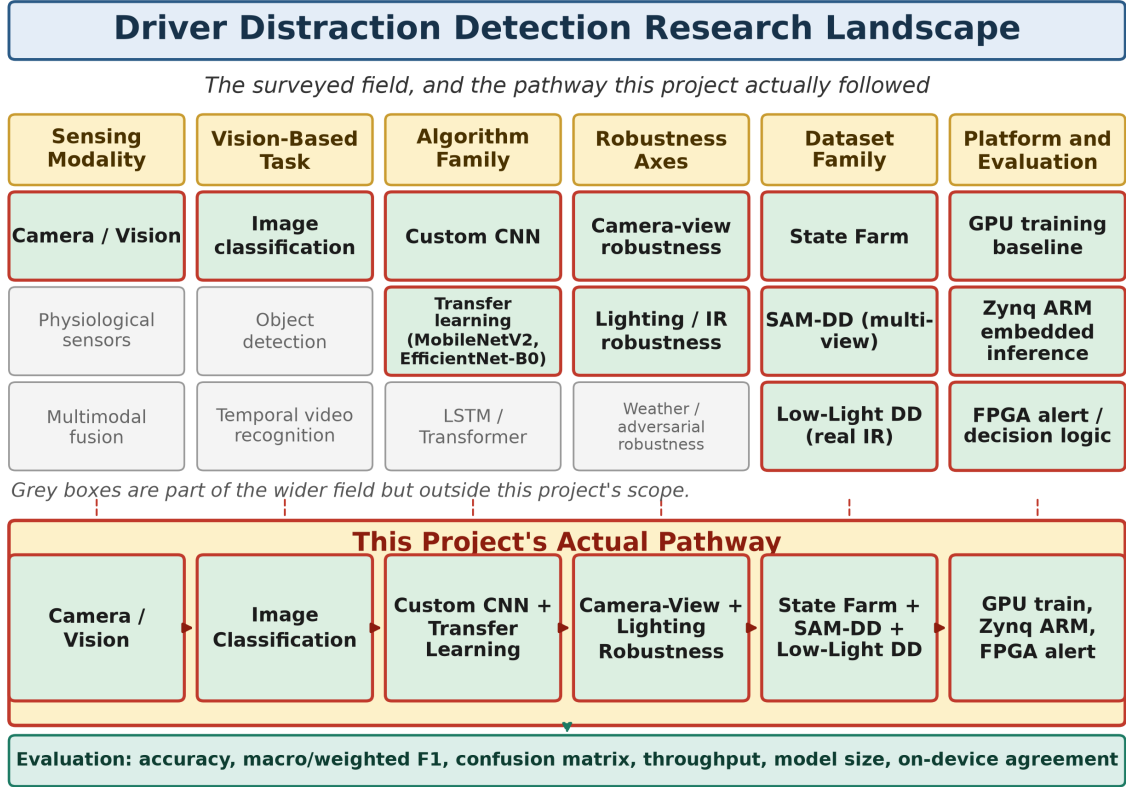


Figure 2.4: The surveyed research landscape and this project's pathway through it, produced during the systematic survey that preceded the practical work.

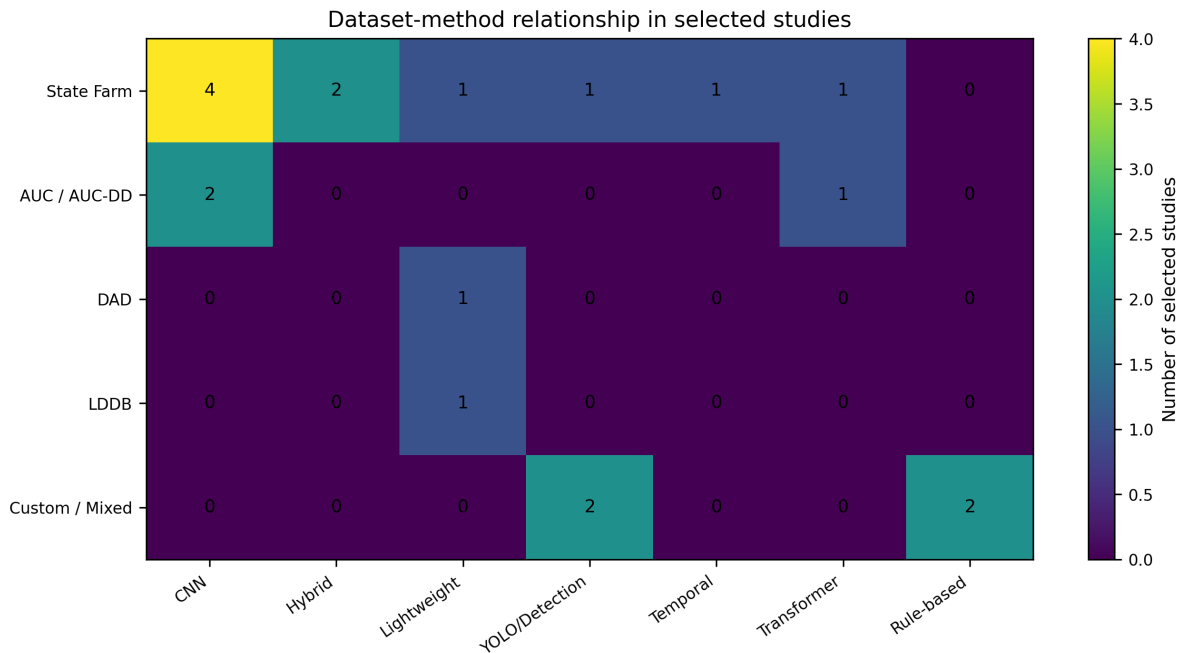


Figure 2.5: Dataset and method coverage across the core studies. Sparse regions indicate combinations that are under-explored rather than combinations that do not work.

Chapter 3

Methodology

3.1 Project management approach

The work proceeded in two phases (Chapter 1). The first one dealt with AI and robustness, covering classification, subject-wise validation and the two robustness axes. The second one was concerned with hardware, covering quantisation and deployment to embedded devices and FPGAs. Progress tracking and documentation were done on a notebook-by-notebook basis in a version-controlled repository, with logging accompanying the code. This systematic approach led to uncovering two true mistakes: data-splitting error (Section 3.2.3) and mismatched preprocessing (Section 4.5.1). The supervisor meetings acted as a project verification point, in particular regarding the choice of including a multi-view stage and continuing with on-device inference after the preliminary diagnosis suggested otherwise.

3.2 Dataset preparation and splitting strategy

3.2.1 Single-view baseline: State Farm

State Farm Distracted Driver Detection dataset (State Farm, 2016) consists of 22,424 labeled images belonging to ten classes (safe driving, four classes of texting/phone use classified by hand, adjusting the radio, drinking, reaching behind, hair/makeup, talking to a passenger) taken with one side-mounted camera from 26 drivers. A naive random image-level split and a subject-wise split leaving out entire drivers were compared directly, with the latter used in all subsequent experiments. The exact splitting code is given in Appendix H, Listing H.1. Random split allows one and the same driver’s face and surroundings to be in both halves of the split, making it even more complicated with almost duplicate video frames in both halves as well. This results in increased accuracy metrics but does not necessarily imply good generalisation to an unseen driver. The class balance in the resulting splits was verified rather than assumed (Figure 3.1).

3.2.2 Multi-view extension: SAM-DD

SAM-DD (Yang et al., 2024) was recorded on an indoor driving simulator with 42 subjects involved. It was chosen due to providing paired front and side images of the same people performing similar actions, which makes it more challenging than a single-view dataset. Simulator origin is an important drawback, evident in metadata rather than hidden.

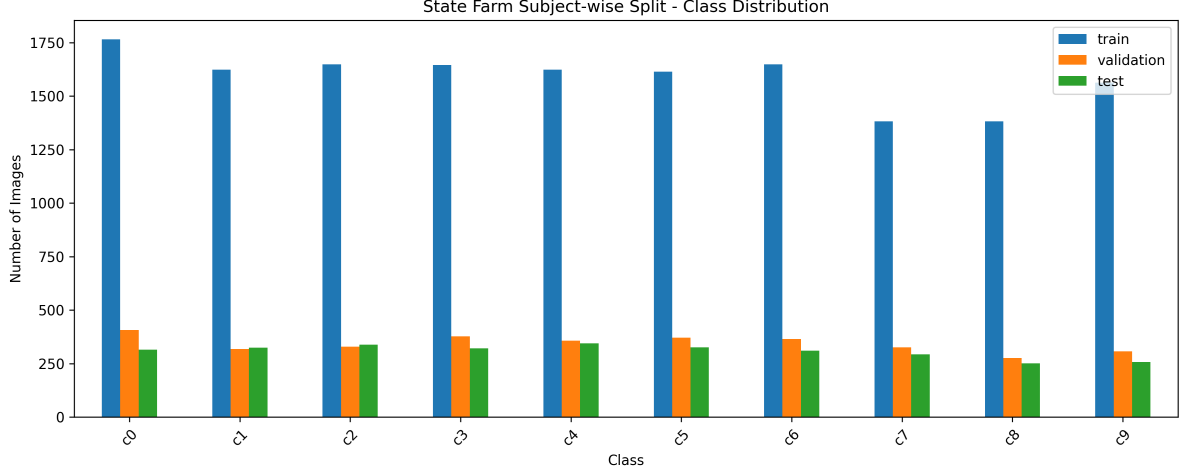


Figure 3.1: Class distribution across the State Farm subject-wise train/validation/test splits.

There is no official class name mapping for SAM-DD (numerically indexed classes). Therefore, the mapping used in this project was created through the visual inspection of sample images along with the confidence level (high, medium or low) assigned to each class. Two SAM-DD classes (adjusting glasses and fatigue/head drop) have no equivalents among State Farm’s classes and had to be completely omitted rather than put in the closest class, thus introducing silent mislabelling.

A common metadata format (image_path, label_id, label_name, subject_id, dataset_name, camera_view, vehicle_id, lighting, modality, split, is_synthetic, notes) was adopted in order to include any dataset without changing the pipeline. In a bidirectional case, special care had to be taken regarding left and right classes, since the front camera mirrored the image with respect to the side-mounted one.

3.2.3 Lighting extension: the Low-Light Driver Distraction Dataset

The lighting experiment used the Novel Driver Distraction Dataset with low-light support (Saad et al., 2020; Hussien et al., 2020). It contains 52,350 images from 70 drivers using the same ten-class taxonomy as State Farm. Although it is described as a low-light dataset, inspection showed that only 9 drivers, contributing 6,750 images, were recorded at night. The remaining 61 drivers were recorded during the day using the same infrared-capable camera. This was established using mean greyscale brightness with a threshold of 40 and confirmed by manually inspecting samples from more than eighteen drivers.

Four classes retained an unresolved hand-side ambiguity: "texting_left", "texting_right", "phone_left" and "phone_right". The side-profile images did not always make the driver’s left and right hands distinguishable. These classes were retained because removing them would change the shared ten-class task, but the ambiguity is considered when interpreting the class-level results.

A split error was also found during preparation. The first split placed none of the nine night drivers in the test partition, meaning that an apparent night evaluation contained no night subjects. The correction was to split the day and night driver pools independently using the same 70/15/15 proportions. This ensured that night drivers were represented in the training, validation and test partitions.

The final lighting experiment combined State Farm daytime data with only the 6,750 real night images. The other 45,600 daytime images from the infrared-capable camera were excluded because including them would mix lighting and sensor-modality changes in the same comparison.

3.2.4 Subject-wise integrity across every dataset and stage

Since subject-wise evaluation lies in the basis of all figures in this report, Table 3.1 shows the split for each dataset and stage. The unit of splitting is an individual driver; there is no subject repeated in several splits and all test statistics from Chapter 4 are calculated based on the held-out subjects, with no reuse of either training nor validation data. Splits were made once per dataset and then kept as is, so that cross-view and lighting experiments are evaluated on the same held-out State Farm drivers as the baseline. Driver names are listed in Appendix B.

Table 3.1: Subject-wise splitting applied consistently across all datasets and stages. No subject appears in more than one partition and all reported test figures come from the held-out subjects listed here. The Low-Light daytime rows are excluded from training for the reason given in Section 3.2.3.

Dataset	Used in	Split unit	Subjects (tr/val/te)	Images (tr/val/te)
State Farm	all three stages	driver	18 / 4 / 4	15,899 / 3,439 / 3,086
SAM-DD (both views)	cross-view	subject	29 / 6 / 7	64,312 / 16,324 / 14,906
Low-Light DD (night)	low-light	driver	6 / 1 / 2	4,500 / 750 / 1,500
Low-Light DD (day)	excluded	driver	42 / 9 / 10	31,425 / 6,675 / 7,500

The unified table holds 170,316 rows. The cross-view experiment draws 80,211 training images and 17,992 test images from it and the lighting experiment draws 20,399 and 4,586.

3.3 Model selection and training

Three architectures were compared: a custom CNN used as a from-scratch reference, MobileNetV2 (Sandler et al., 2018) and EfficientNet-B0 (Tan and Le, 2019). The two transfer-learning models used ImageNet-trained weights with newly added classification heads. All three models used the same subject-wise partitions and were evaluated using accuracy, macro F1, weighted F1, per-class precision and recall, confusion matrices, throughput and model size. MobileNetV2 and EfficientNet-B0 were selected because both offer a useful balance between classification performance and deployment cost.

The two robustness experiments followed the same training procedure. The backbone was first frozen while the new classification head was trained for ten epochs. The final 30 MobileNetV2 layers were then unfrozen and fine-tuned for another ten epochs using a learning rate of 1×10^{-5} . This prevented large early updates from damaging the pretrained features before the new head had learned useful representations. Early stopping with a patience of five restored the best validation checkpoint. A fixed random seed improved repeatability, although a single run does not measure run-to-run variation, as discussed in Section 5.6.

3.4 Quantisation and on-device deployment

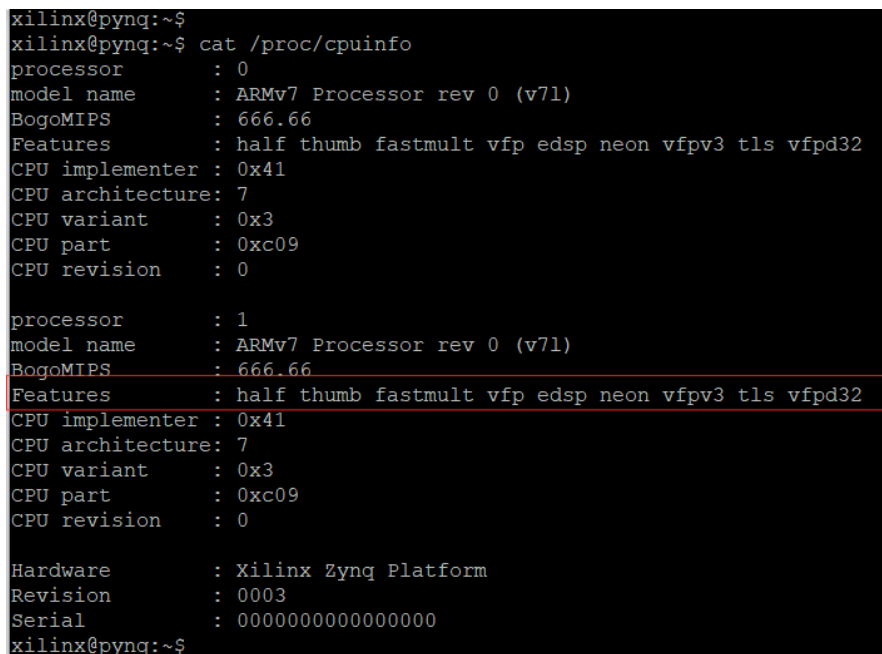
3.4.1 Model conversion

The trained model had to be converted into a lightweight format the board’s processor could run. This needed its own software setup matching the exact version the model was saved under, since the everyday setup could open the file but produced a version the board could not run at all. The fix used is given in Appendix H, Listing H.2.

A second problem showed up during conversion: a layer used only during training kept sneaking unsupported steps into the exported model, even after being switched off, because that layer could not be cleared the normal way in this software version. The fix was to override its behaviour directly so it simply passed its input straight through. This was checked two ways: the exported model contained none of the unwanted steps and gave identical results to the original. The model was then compressed to 8-bit precision, calibrated against 120 sample images (Google, 2026).

3.4.2 Board bring-up and the ARM32 runtime issue

The board, a Zynq-based ARM processor borrowed from the university’s electronics lab, was set up with a community-built Linux image (AMD/Xilinx, 2026), since the manufacturer’s own tools do not support this chip. Bringing it online also fixed two smaller, unrelated software problems. The bigger issue was that the only available version of the software needed to run the model on this device loaded it correctly but crashed the moment it tried to compute a prediction. Memory was ruled out, both versions of the compressed model failed identically and turning off various accelerations made no difference. The real cause was a mismatch between what instructions this processor could actually run and what the published software assumed every chip of its type could do: an older processor missing one math instruction the software relied on.



```
xilinx@pynq:~$  
xilinx@pynq:~$ cat /proc/cpuinfo  
processor       : 0  
model name     : ARMv7 Processor rev 0 (v7l)  
BogoMIPS      : 666.66  
Features      : half thumb fastmult vfp edsp neon vfpv3 tls vfpd32  
CPU implementer : 0x41  
CPU architecture: 7  
CPU variant    : 0x3  
CPU part      : 0xc09  
CPU revision   : 0  
  
processor       : 1  
model name     : ARMv7 Processor rev 0 (v7l)  
BogoMIPS      : 666.66  
Features      : half thumb fastmult vfp edsp neon vfpv3 tls vfpd32  
CPU implementer : 0x41  
CPU architecture: 7  
CPU variant    : 0x3  
CPU part      : 0xc09  
CPU revision   : 0  
  
Hardware       : Xilinx Zynq Platform  
Revision      : 0003  
Serial        : 0000000000000000  
xilinx@pynq:~$
```

Figure 3.2: Confirming directly on the board that this processor is missing the math instruction the published software assumed it would have.

The fix was to rebuild that software from scratch on the board itself, targeting the processor’s real capabilities. This meant building a basic tool from scratch first too, since the board’s software was too old to have one ready-made. Building it natively, rather than elsewhere and copying it across, let the process detect exactly what the chip supports rather than guessing. This solved the crash, confirmed by comparing predictions directly against the reference version (Section 4.5.1).

3.5 FPGA alert-controller design and verification

A full hardware version of the classifier was considered early on, in line with a fallback plan, but judged unlikely to reach a working state in the time available (Lu et al., 2025; Parra et al., 2023). Instead, the hardware work focuses on the stage right after the classifier: turning a raw prediction into an alert. This was built as a small circuit that only raises an alert after eight confident "distracted" readings in a row and clears it after eight confident "safe" readings in a row, matching the same cut-off already proven in the software version. Reusing an already-working idea in hardware, rather than a separate rule, keeps the design grounded in something already shown to work.

This was checked two ways. A standalone test ran the logic against ten made-up edge cases and 28 real predictions from the trained model, including two real mistakes it made, were replayed through a simulated version of the full link between camera, processor and alert circuit, to check the design handles real, imperfect data. The circuit was then built for a low-cost board, chosen deliberately as an affordable target, using Vivado 2017.4 in batch mode. The full design is in Appendix H, Listing H.3.

3.6 Alert delivery, tools and test strategy

To turn a detection into something a driver or fleet manager would see, the board runs a continuous capture-classify-alert loop, using the same logic as the hardware version above. Every alert, with the photo that triggered it, is sent to a small server, which notifies a phone app. A buzzer wired to the board gives a second, independent alert at the same moment, so a driver is warned even without a phone nearby. This needed a small custom hardware addition, built, tested and loaded onto the board, exposing a control pin with no way of being switched on before (Xilinx, 2019) - the one part of this project’s hardware work tested on real equipment rather than estimated (Section 4.6). Full detail and the three layers the alert path was checked in, are in Appendix G and Appendix F.

The AI side used Python and standard machine-learning tools; the hardware side used the manufacturer’s own design software and the board’s Linux environment. Testing happened at every stage, not just the end. Each dataset was checked immediately after being added, for missing labels, lost images, or a driver appearing in more than one split, exactly what caught a real bug early on. Every accuracy figure comes from data the model never saw during training. The conversion was checked both structurally and by confirming matching output and the same standard was applied on the board, checking predictions actually matched rather than assuming a crash disappearing meant everything was correct.

3.7 Ethical, legal, professional and social issues

Formal ethics approval was not required because the experiments used existing public datasets (State Farm, 2016; Yang et al., 2024; Hussien et al., 2020), together with limited test footage recorded by the researcher. No participants were recruited and no third-party personal data were collected. The datasets were stored locally rather than redistributed, in accordance with their licences; one dataset is restricted to non-commercial use.

The available datasets do not provide sufficient demographic information to assess performance fairly across age, ethnicity or body type. No claim of demographic fairness is therefore made. Driver monitoring also creates a privacy concern because a camera continuously observes the driver. Local processing reduces this risk because ordinary footage remains on the device, although the current prototype sends the image associated with an alert. A practical product should allow that feature to be disabled and should apply clear retention and access controls.

Both missed distractions and false alerts have safety consequences. Repeated false alerts may cause users to ignore or disable the system, which is why the decision logic requires several confident predictions before raising an alert. In line with the BCS Code of Conduct (BCS, The Chartered Institute for IT, 2026), the system is presented as a research prototype rather than a finished safety product. Access to borrowed hardware also constrained the work, since experiments that risked damaging the board could not reasonably be attempted.

Chapter 4

Quality and Results

All results reported in this chapter come from the fixed groups of held-out subjects listed in Table 3.1. Full per-class precision, recall and F1 tables for all six trained models are provided in Appendix C.

4.1 Single-view baseline: model comparison (RQ1)

Table 4.1 compares the three models using the State Farm subject-wise test set. MobileNetV2 achieved the highest test accuracy and macro F1, at 84.67% and 84.01%, respectively. EfficientNet-B0 reached 82.47% accuracy and 81.58% macro F1, while the custom CNN reached 75.60% and 75.27%. EfficientNet-B0 also showed a sizeable validation-to-test difference, at 72.23% versus 82.47%, suggesting that its four-driver validation partition was more difficult than its test partition. Because each partition contains only a small number of drivers, these results should be interpreted cautiously, as discussed in Section 5.6.

Table 4.1: Single-view model comparison on the State Farm subject-wise test set (3,086 images, 4 held-out drivers).

Model	Val. acc.	Test acc.	Macro F1	Weighted F1	Train time (min)
Baseline CNN	60.89%	75.60%	75.27%	76.57%	11.87
MobileNetV2	82.12%	84.67%	84.01%	84.94%	32.03
EfficientNet-B0	72.23%	82.47%	81.58%	82.79%	37.57

The deployment measurements in Table 4.2 and Figure 4.3 were collected on the development laptop, not on embedded hardware. The corresponding on-device comparison is provided in Section 4.4.3. MobileNetV2 was slightly slower than the custom CNN, at 16.64 FPS compared with 17.61 FPS, but improved accuracy by approximately nine percentage points. This accuracy gain justified its selection despite the modest throughput difference.

Table 4.2: Deployment-oriented measurements on the development laptop (GPU-accelerated). These are laptop figures and are not comparable with the on-device figures in Table 4.6.

Model	Size (MB)	Inference (ms)	Throughput (FPS)
Baseline CNN	4.96	56.79	17.61
MobileNetV2	20.78	60.08	16.64
EfficientNet-B0	15.75	64.51	15.50

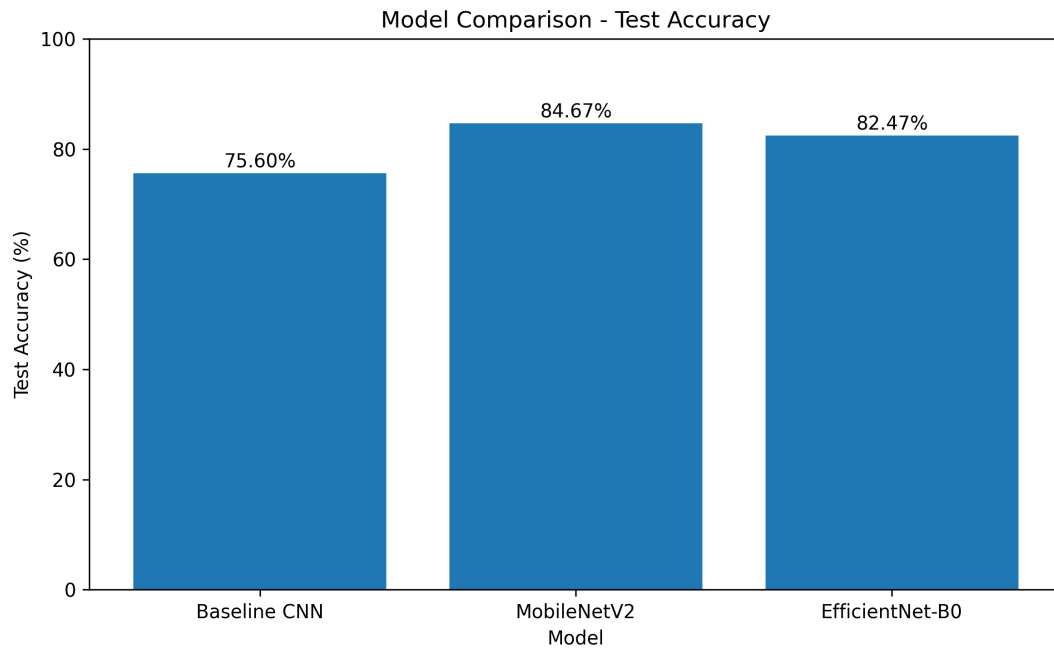


Figure 4.1: Test accuracy of the custom CNN, MobileNetV2 and EfficientNet-B0 on the State Farm subject-wise test set.

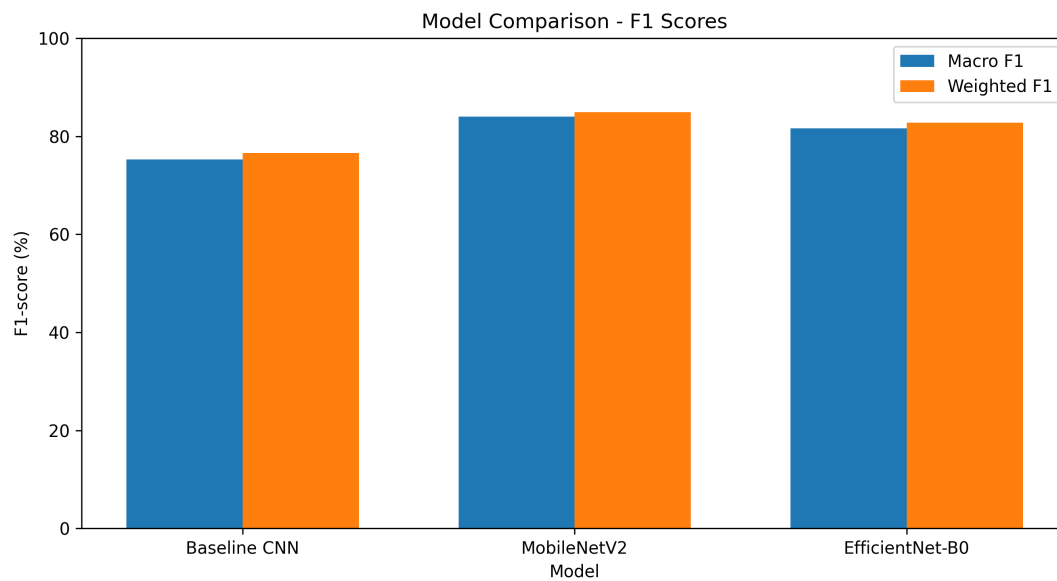


Figure 4.2: Macro and weighted F1 scores of the three single-view baseline models.

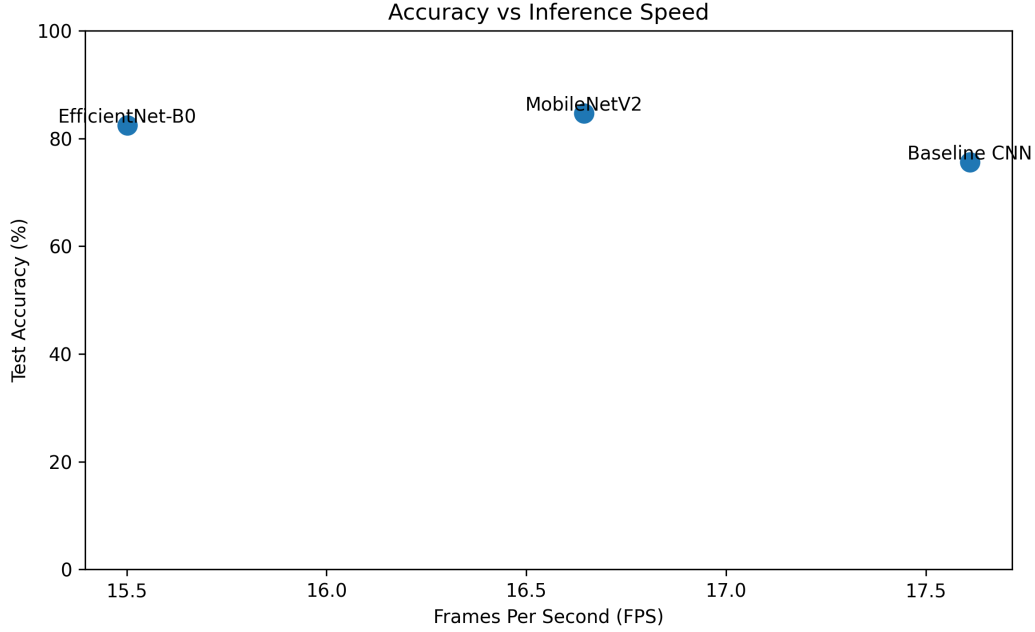


Figure 4.3: Accuracy against laptop throughput for the three baseline models, showing the trade-off underlying the MobileNetV2 selection.

4.2 Cross-view generalisation (RQ2)

MobileNetV2 was retrained on the combined State Farm and SAM-DD table (80,211 training images spanning side and front) with the backbone frozen. Mixed-view accuracy reached only 75.94% and it was clearly uneven, 78.79% on front against 73.93% on side. A model not specifically trained for the view robustness is not automatically robust to the changes of the view, even if this view is present in the training data and is managed equally well by default.

Unfreezing the last 30 layers and fine-tuning at 1×10^{-5} improved every measure in Table 4.3. Early stopping retained epoch 5, where validation accuracy reached its maximum of 91.24%. Training accuracy had already exceeded 98% by epoch 3 of fine-tuning, while the later validation results did not improve, indicating overfitting (Figures 4.4 and 4.5).

Table 4.3: Cross-view MobileNetV2 accuracy, frozen-backbone against fine-tuned, on the 17,992-image held-out mixed-view test set.

Test partition	Frozen-backbone	Fine-tuned	Change
Mixed-view (17,992 images)	75.94%	91.76%	+15.8 pt
side view (10,539 images)	73.93%	91.05%	+17.1 pt
front view (7,453 images)	78.79%	92.77%	+14.0 pt

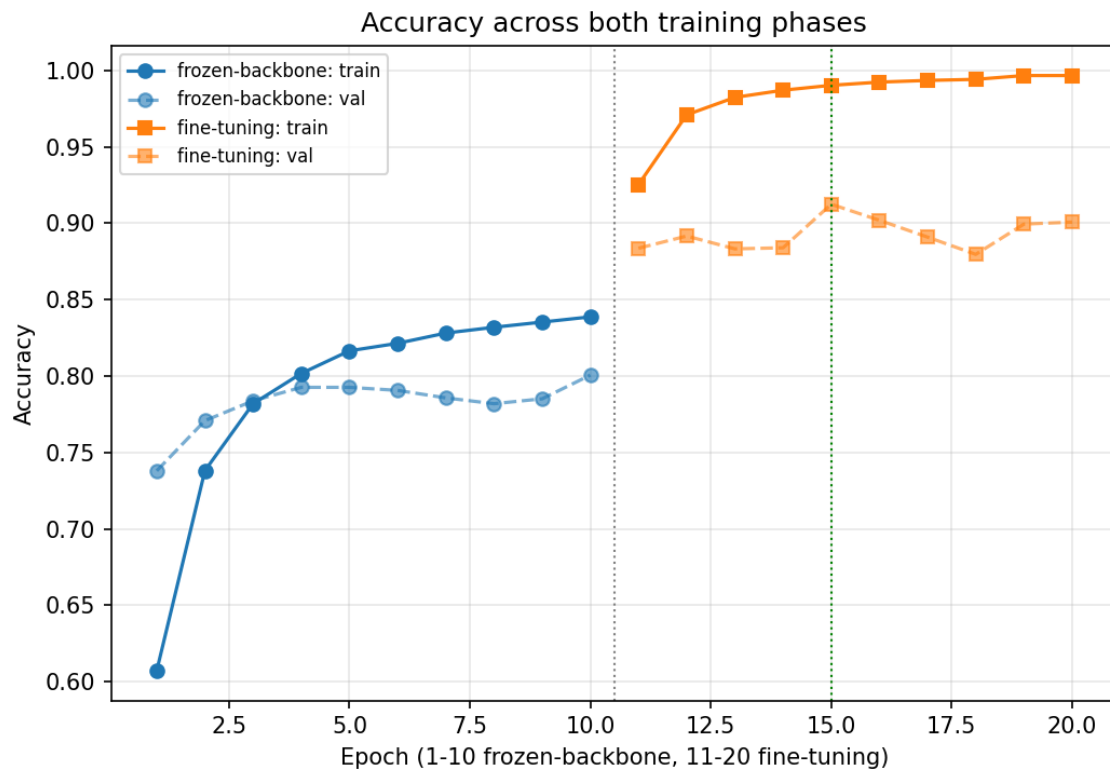


Figure 4.4: Cross-view MobileNetV2 training and validation accuracy across the frozen-backbone and fine-tuning phases.

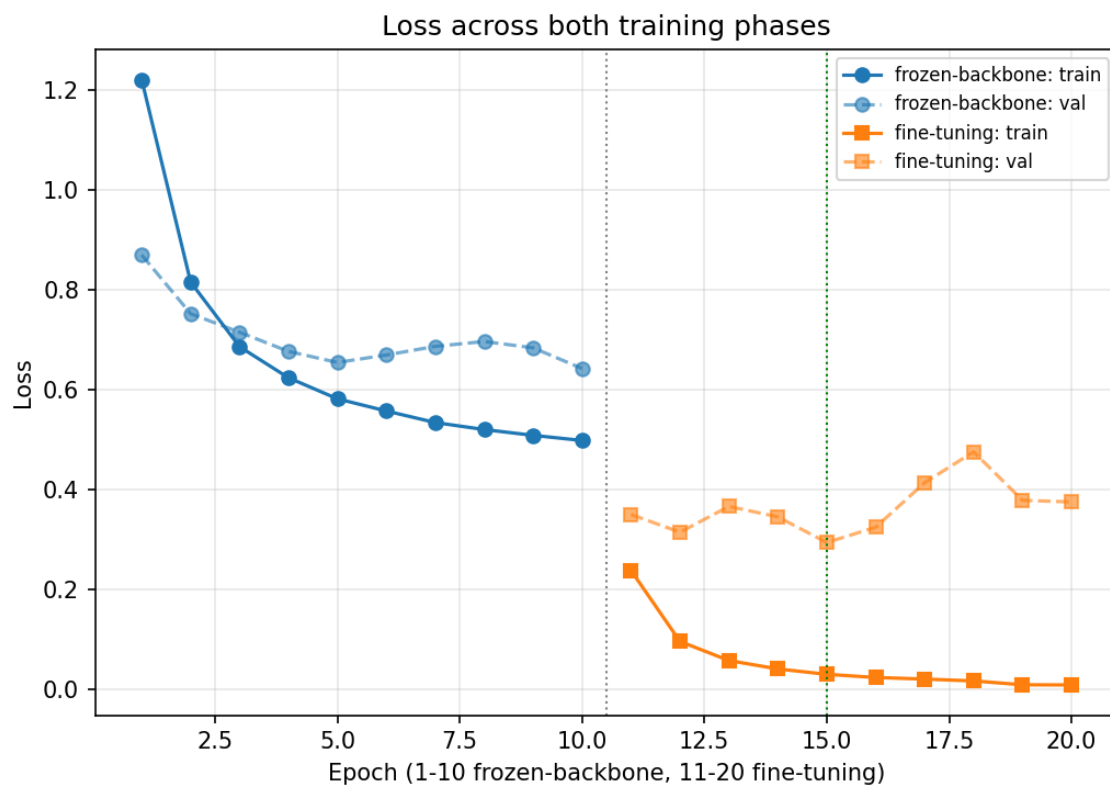


Figure 4.5: Cross-view MobileNetV2 training and validation loss across the frozen-backbone and fine-tuning phases.

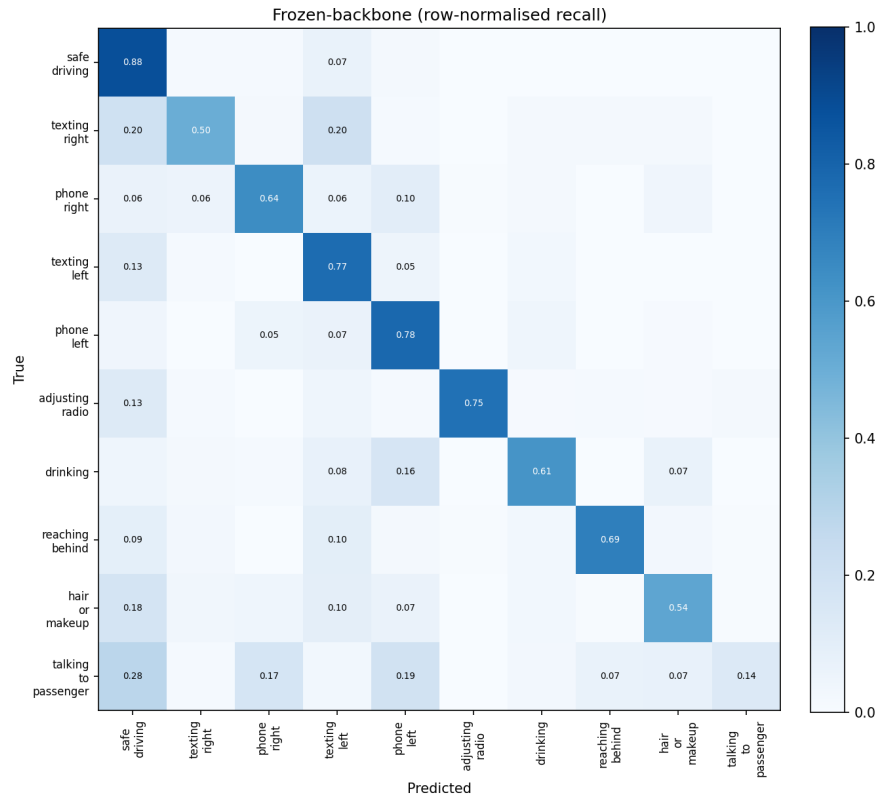


Figure 4.6: Row-normalised confusion matrix for the frozen-backbone cross-view model on the held-out mixed-view test set.

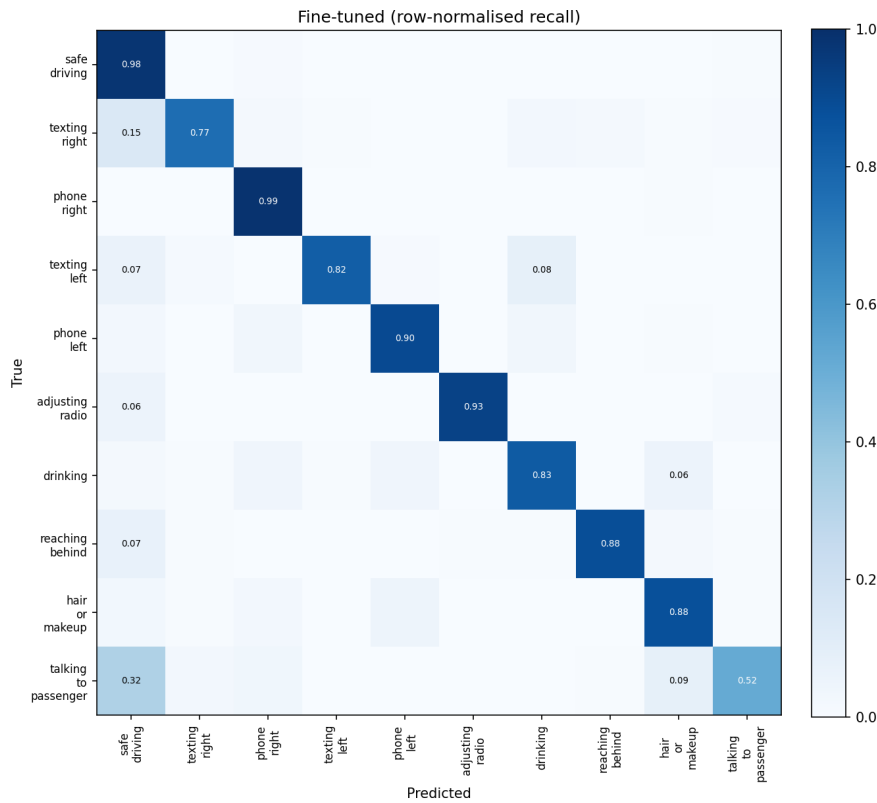


Figure 4.7: Row-normalised confusion matrix for the fine-tuned cross-view model on the held-out mixed-view test set.

All ten classes improved, but unevenly and this unevenness is actually the more useful result (Section 4.7). Figure 4.8 shows predictions on real held-out images from both views.



Figure 4.8: Qualitative predictions from the fine-tuned cross-view model on held-out test images spanning both camera views.

4.3 Lighting-domain robustness (RQ3)

This stage tests the performance of the model trained during the day on night-time video data recorded in active IR light, instead of the day video darkened artificially. The results of this stage are the most extreme throughout the whole project.

4.3.1 The day-only baseline collapses

The best single-view State Farm MobileNetV2 checkpoint, the same model that got 84.67% on its own daytime test set, was directly tested on the 1,500 held-out real night images without any retraining. Its night accuracy is 19.53

This number misrepresents the extent of the problem. Instead of showing degradation of performance on ten classes, the model collapsed. It assigns nearly all night images to two classes, with 99.3% recall on `texting_left` and 96.0% recall on `reaching_behind` and 0.0% recall on all other eight classes (Appendix C). Thus, the model did not just become confused, it stopped

discriminating between classes, which may be considered even more harmful for an alerting stage downstream that receives confident misclassifications rather than just noisy data.

4.3.2 Training on real night data recovers most of the gap

A fresh MobileNetV2 was trained on the joined table of 20,399 training images (State Farm daytime images, plus only real night subset) for the reasons outlined in Section 3.2.3. The training process matched the cross-view stage to make comparison easy.

Table 4.4: Lighting-domain robustness. The baseline column is the day-trained model applied to night images with no retraining. All test figures come from held-out drivers.

Test partition	Day-only baseline	Frozen-backbone	Fine-tuned
Mixed day+night (4,586 images)	-	62.30%	77.91%
day (3,086 images)	84.67%	64.55%	81.21%
night (1,500 images)	19.53%	57.67%	71.13%

First, training a new classification head on real day and night images with backbone pre-trained on ImageNet improved the night accuracy from 19.53% to 57.67%. Thus, the collapse seems to be mostly caused by the classification head rather than a deficiency of ImageNet features for infrared illuminated images.

Second, fine-tuning of the whole model helped to recover most of the lost performance, giving the model night accuracy of 71.13%. The improvement of 51.6 percentage points was obtained on real captured video frames rather than synthetically produced approximations.

Third, there is a limit to how much can be gained in this way. The daytime accuracy of fine-tuned model reached only 81.21%, 3.5 percentage points below the 84.67% obtained by original single-domain model on the same test set. This means that training one shared classification head for two lighting domains comes at a cost of single-domain accuracy.

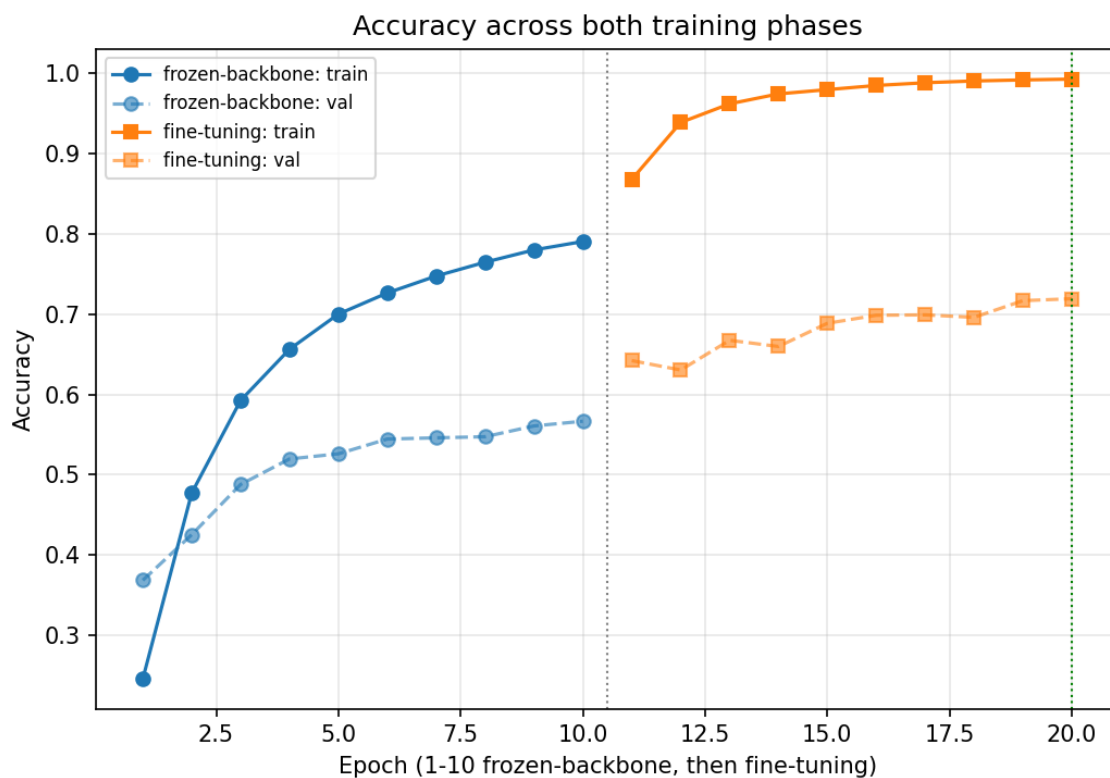


Figure 4.9: Low-light MobileNetV2 training and validation accuracy across the frozen-backbone and fine-tuning phases.

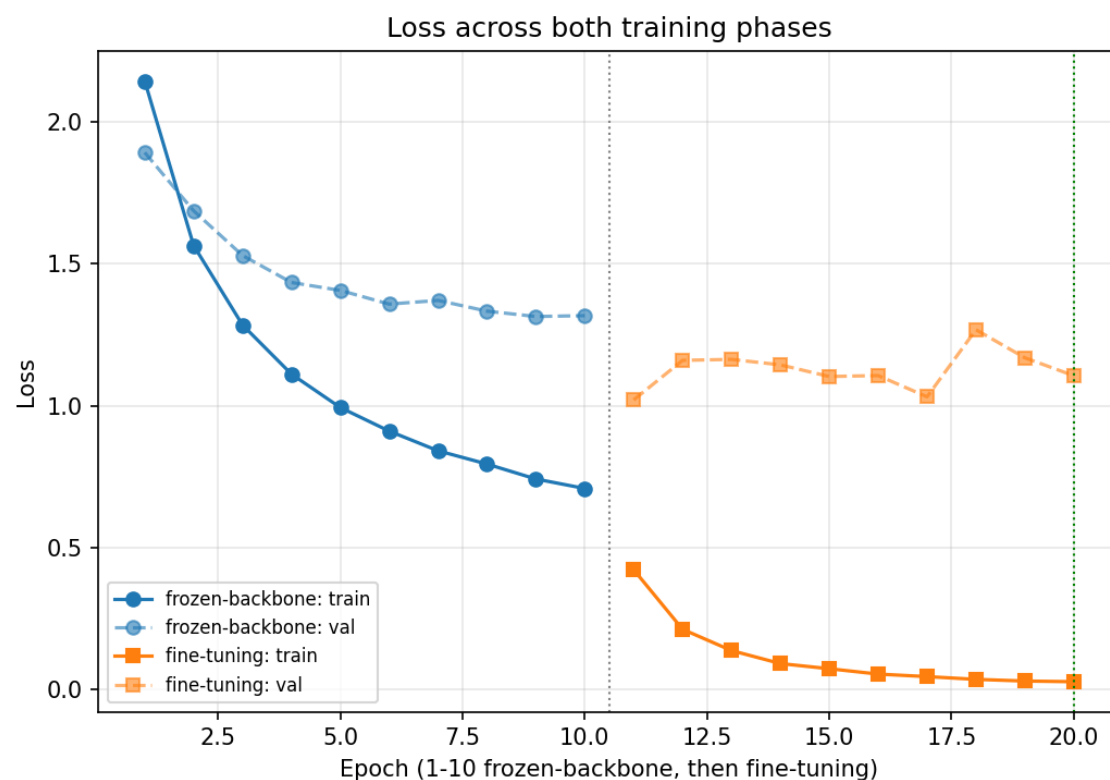


Figure 4.10: Low-light MobileNetV2 training and validation loss across the frozen-backbone and fine-tuning phases.

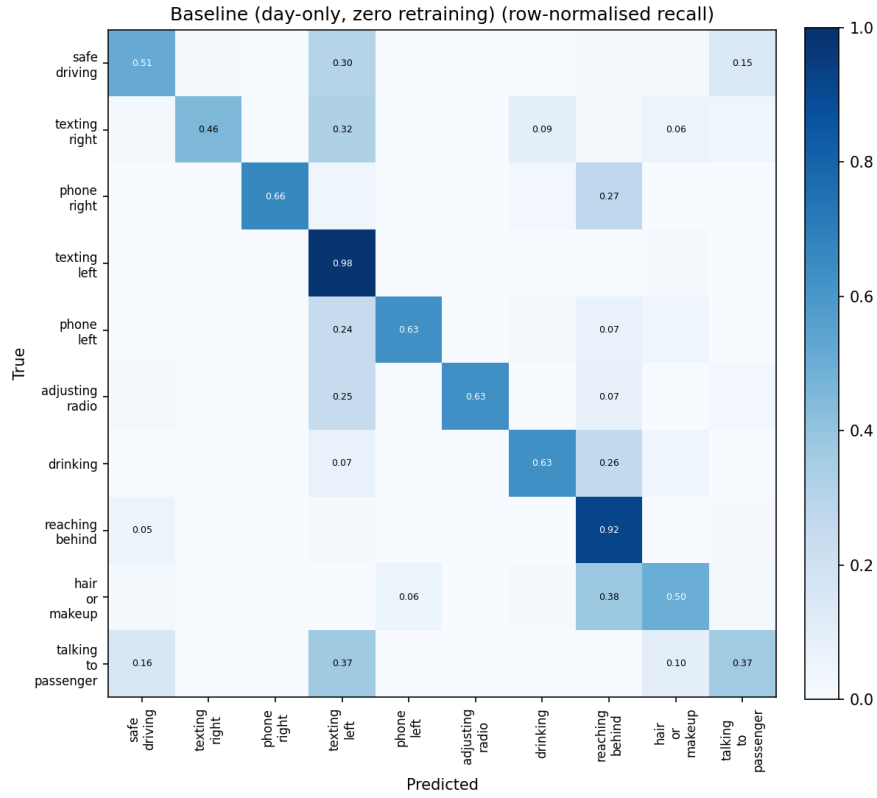


Figure 4.11: Row-normalised confusion matrix for the day-trained MobileNetV2 evaluated directly on the 1,500 held-out real night images. The predictions collapse mainly into two classes.

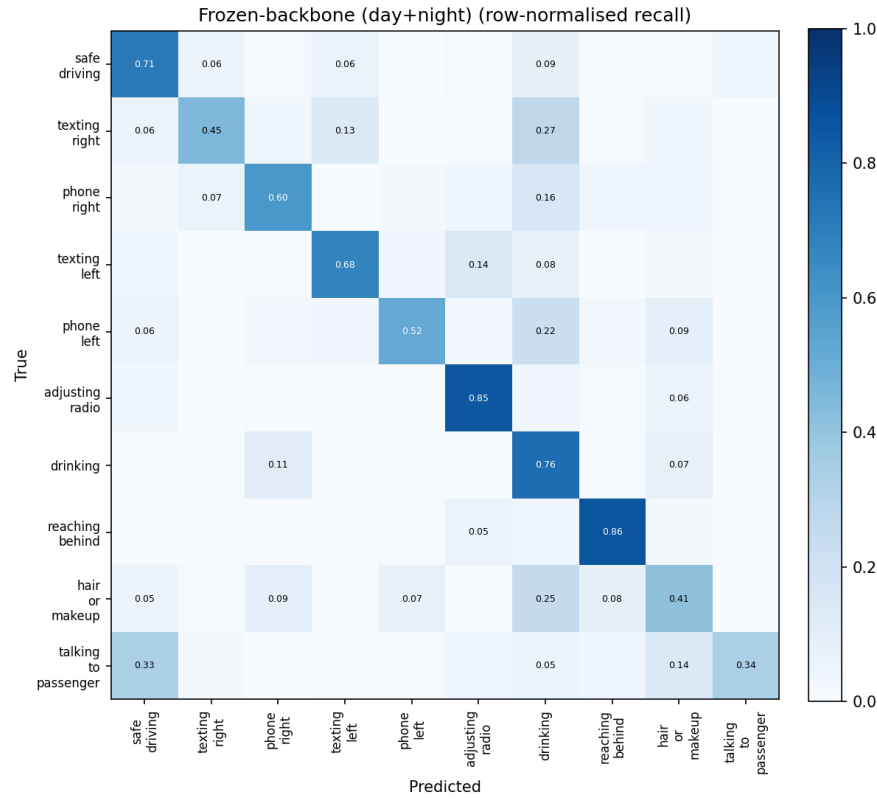


Figure 4.12: Row-normalised confusion matrix for the frozen-backbone low-light model on the held-out mixed day-and-night test set.

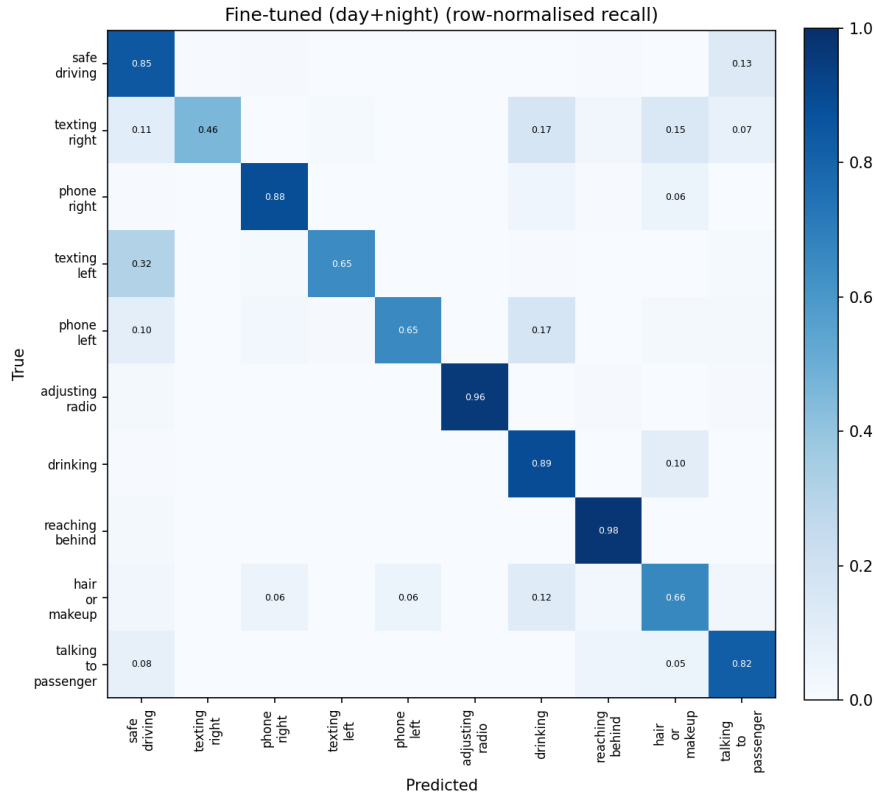


Figure 4.13: Row-normalised confusion matrix for the fine-tuned low-light model on the held-out mixed day-and-night test set.



Figure 4.14: Qualitative predictions from the fine-tuned low-light model on held-out real night-time images.

4.4 Quantisation: accuracy, size and latency (RQ4)

Quantisation is what makes embedded deployment possible at all. Its cost is reported here across all four measures (accuracy, size, latency and throughput) before and after conversion.

4.4.1 Model size

The fine-tuned cross-view Keras model takes 21,791,872 bytes (20.78 MB) in size, while the INT8 TensorFlow Lite conversion yields 2,723,448 bytes (2.60 MB), which gives the compression ratio of **8.00** $\times$. This corresponds to the expected result of INT8 quantisation applied to mostly float32 model and demonstrates that the conversion happened as it should and did not preserve the floating point weights in disguise.

4.4.2 Accuracy

Two before-and-after comparisons were measured on different samples and both are reported here because the difference between them is informative on its own (Table 4.5).

Table 4.5: Quantisation accuracy cost, measured on different held-out samples. The two loss figures differ because INT8 quantisation error is data-dependent, not because either measurement is unreliable.

Model	Evaluation sample	Keras float32	TFLite INT8	Cost
Cross-view, frozen	500 mixed-view images	76.80%	76.00%	−0.8 pt
Cross-view, frozen	300 random side images	59.30%	54.70%	−4.6 pt
Cross-view, fine-tuned	300 random side images	80.00%	77.30%	−2.7 pt

The first comparison used a 500-image mixed-view sample and found an accuracy reduction of 0.8 percentage points. The second used an unbalanced random sample of 300 side-view images and found a 4.6-point reduction for the same frozen model. These figures are not directly comparable because the samples differ in size and composition. The float32 model itself achieved only 59.3% on the 300-image sample, well below its 73.93% result across the complete side-view test partition, showing that this was a more difficult sample.

The deployed fine-tuned model lost 2.7 percentage points on the same 300-image sample, compared with 4.6 points for the frozen model. One possible explanation is that the better-trained model produced larger prediction margins and was therefore less sensitive to quantisation noise, although this was not tested directly. All measured reductions remain within the few-percentage-point range discussed by Abushahla et al. (2025).

4.4.3 The consolidated software-to-hardware comparison

Table 4.6 brings together the entire processing pipeline into one comparison: the laptop float32 reference, the laptop-based INT8 conversion and the same model run on actual ZC702 hardware, each with its respective FPGA fabric usage shown alongside. The goal of the table is to clearly distinguish between laptop and hardware performance.

Table 4.6: Consolidated software, deployment and hardware comparison for the fine-tuned cross-view MobileNetV2 and the FPGA alert controller. Laptop and on-device figures are reported separately and are not interchangeable.

Stage	Platform	Model size	Accuracy	Latency / throughput
Reference (float32)	Laptop (RTX 3060)	20.78 MB	80.0%	60.08 ms / 16.64 FPS
Quantised (INT8)	Laptop (converted)	2.60 MB	77.3% ^c	not separately timed
Deployed (INT8)	ZC702 (Cortex-A9)	2.60 MB	77.3%	9,150 ms / 0.11 FPS

Quantisation results in eight times smaller model size with an accuracy cost of 2.7 percentage points. Migration from laptop to target involves no accuracy loss, because the INT8 model behaves identically on both platforms, but brings a speed penalty of around 150 times. The fabric alert-decision stage remains free of cost almost completely, with the entire pipeline cost being in the CNN. This is what justifies the engineering goal of developing a dedicated CNN accelerator (Section 5.7).

4.5 On-device deployment on the ZC702 (RQ4)

4.5.1 Establishing runtime correctness first

Any accurate on-device figure had to prove numerical correctness in addition to being crash-proof, before the trustworthiness of the number could be established. An initial comparison of the board with the laptop revealed only 71% agreement, implying an incorrect build at first glance. The source of error was in the test script, which resizes images via `PIL.Image.resize()` during training, while the on-device deploy uses `cv2.resize` function, leading to a difference. After preprocessing harmonisation, agreement became 100 out of 100 predictions, showing identity of results. This means that the rebuilt ARM32 runtime is numerically equivalent to the official implementation, a stronger statement than crash avoidance alone.

4.5.2 Accuracy on real hardware

A random selection of 300 images was tested in four different settings: Keras float32 and on-device TFLite INT8, both for the frozen and fine-tuned checkpoints (Figure 4.15). Fine-tuning increased on-device accuracy from 54.7% to 77.3%, for a gain of 22.6 percentage points, observed on real hardware. It implies that the laboratory gains survive quantisation and deployment, not merely assumed.

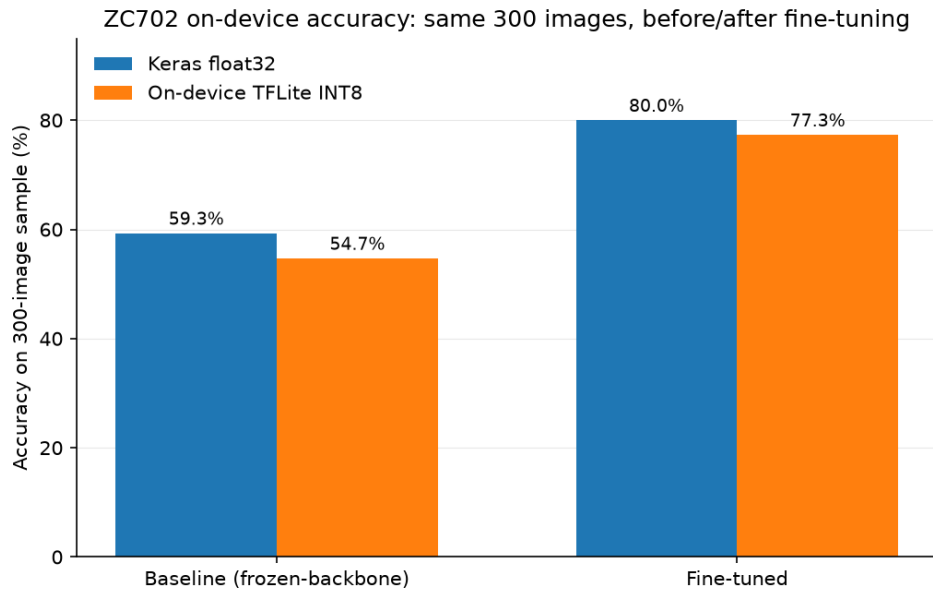


Figure 4.15: On-device (ZC702, TFLite INT8) against reference (Keras float32) accuracy on the same 300-image held-out sample, before and after fine-tuning.



Figure 4.16: A real inference run on the ZC702, showing the predicted class and confidence reported in Section 4.5.2.

Figure 4.16 shows this measured directly on the board, not estimated from the laptop figure.

Inference on the device took about 9.15 seconds per image, without XNNPACK acceleration, which represents the configuration that is known to be safe, as validated under the ARM32 crash fix, being the most explicit limitation of this deployment. Section 5.4 covers the reasons why re-enabling XNNPACK was considered but not attempted.

4.6 FPGA alert-controller verification and synthesis (RQ5)

All testbenches pass without issues. The standalone hysteresis testbench passes all ten checks, while the end-to-end testbench replays 28 real classifications over a simulated UART link, including two real model misclassifications, both of which are correctly ignored without a false positive alert. It matches the designed failure mode of the system, validated against real erroneous model output and not artificial noise. Out-of-context synthesis for the Artix-7 target (Table 4.7, Figure 4.17) shows that the controller uses only a tiny fraction of the device and satisfies the 100 MHz timing requirement with 7.276 ns of slack.

Table 4.7: Post-synthesis results for the alert controller, `xc7a35tcpg236-1`, out-of-context. Raw report excerpts are in Appendix E.

Metric	Value
Slice LUTs	18 (0.09% of 20,800)
Slice registers (flip-flops)	13 (0.03% of 41,600)
Block RAM	0
DSP slices	0
Worst negative slack at 100 MHz	+7.276 ns
Worst hold slack	+0.272 ns
Failing timing endpoints	0 of 17
Implied theoretical maximum frequency	~367 MHz
Estimated total on-chip power	0.069 W (0.068 W static)

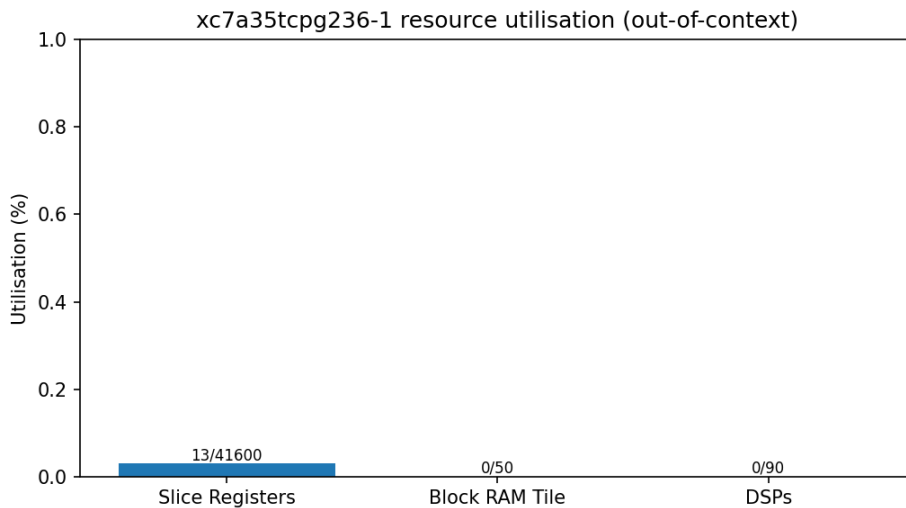


Figure 4.17: Post-synthesis resource utilisation for the alert controller on `xc7a35tcpg236-1`.

First, it should be stressed that these are post-synthesis estimates, rather than measurement of the placed-and-routed design on the actual silicon, as access to the real Artix-7 board was not

possible and this target was selected as the representative example of a low-cost device. Vivado report shows that the actual LUT count after physical optimisations is usually less; thus, the current estimate is conservative, although still an estimate. Second, the power estimate is made at the medium confidence level, without the vector simulation. Third and most importantly, the above figures are related to the decision stage only, not the CNN, as the classifier runs in software on the ARM processor and is not covered by the synthesised design. Hence, nothing in Table 4.7 should be seen as a cost estimate of MobileNetV2 in fabric.

The work on FPGA demonstrates a narrower but still concrete achievement: the logic that transforms the stream of noisy classifications into a consistent set of alerts is low-cost and low-power, implementable on the smallest FPGA devices and verified against real model output, including actual mispredictions, with the reproducibility guaranteed by the provided scripts.

4.7 Class-level error analysis across all three stages

A summary of just the accuracy of the headline would hide the more revealing structure that exists in these results. Table 4.8 captures the most illuminating classes.

Table 4.8: Selected per-class results across stages (P = precision, R = recall). Full tables in Appendix C.

Class	Cross-view frozen		Cross-view fine-tuned		Low-light fine-tuned	
	P	R	P	R	P	R
talking_to_passenger	0.61	0.14	0.60	0.52	0.72	0.82
texting_left	0.42	0.77	0.97	0.82	0.97	0.65
texting_right	0.69	0.50	0.92	0.77	0.99	0.46
hair_or_makeup	0.70	0.54	0.87	0.88	0.60	0.66
drinking	0.75	0.61	0.84	0.83	0.64	0.89
safe_driving	0.89	0.88	0.94	0.98	0.55	0.85

The "hair_or_makeup" class remained difficult across the project. It had the lowest F1 in the single-view baseline at 0.71 and in the frozen low-light model at 0.42. Gestures near the face resemble "phone_to_ear", while the relevant object is often small or partly hidden, making it especially vulnerable to reduced image quality.

In the cross-view experiment, recall for "talking_to_passenger" improved from 0.14 to 0.52 after fine-tuning. However, this class contained only 258 test images compared with 9,006 for "safe_driving", so its recall is less stable than the headline mixed-view result suggests.

The clearest fine-tuning effect appears in "texting_left". Precision increased from 0.42 to 0.97 while recall remained broadly similar, increasing from 0.77 to 0.82. The gain therefore came mainly from reducing incorrect "texting_left" predictions rather than simply predicting the class more often.

The low-light result for "texting_right" shows the opposite pattern, with precision of 0.99 but recall of 0.46. This is consistent with the unresolved left-right labelling ambiguity discussed in Section 3.2.3, although the current evidence cannot separate label noise from a genuine model weakness.

4.8 Summary against the research questions

RQ1 - Model comparison.

MobileNetV2 beats both other options in subject-wise evaluation (84.67% accuracy, 84.01% macro F1) while retaining the best deployment properties in size and throughput (Section 4.1).

RQ2 - Cross-view robustness.

A simple multi-view model reveals a real but uneven gap (75.94% mixed, **side** 4.9 points behind **front**). Fine-tuning closes most of this gap, reaching 91.76%, with evidence from class scores showing that gains come from improved class boundaries rather than a uniform shift (Section 4.2, Section 4.7).

RQ3 - Lighting robustness.

A day-trained model fails at 19.53% on real infrared-illuminated night footage, scoring zero recall on eight out of ten classes. Night training recovers this to 71.13%, but loses 3.5 points of daytime accuracy (Section 4.3).

RQ4 - Quantisation and deployment cost.

INT8 quantisation reduced the model size by $8\times$ at a cost of 2.7 accuracy points on the deployment sample. The embedded runtime reproduced the laptop INT8 predictions, but the ZC702 required approximately 9.15 seconds per image, making it around $150\times$ slower than the laptop measurement (Section 4.4, Section 4.5).

RQ5 - What is credibly implementable in fabric.

The decision stage, rather than the classifier, was the part that could be implemented and verified credibly within the available time. Testing used real model predictions, including misclassifications. Post-synthesis results showed 18 LUTs, 13 flip-flops and 7.276 ns of positive timing slack. These remain post-synthesis estimates rather than measurements from physical Artix-7 hardware (Section 4.6).

Chapter 5

Evaluation and Conclusion

5.1 Final evaluation against objectives

Both objectives were met, although the implementation changed as evidence and practical constraints emerged.

The first objective was to evaluate camera-view and lighting robustness using suitable real data. The multi-view experiment proceeded as planned. For lighting, the original proposal to generate infrared-like images was replaced with real infrared-illuminated night footage once it became available. This removed the main limitation of using a visual approximation and produced a direct measurement of the day-to-night domain shift.

The second objective was to demonstrate a low-cost embedded detection-and-alert pipeline. The CNN was deployed on the ZC702's ARM processor, while the FPGA work focused on the decision stage rather than a complete CNN accelerator. This reduced the hardware scope, but allowed both stages to be tested with real model outputs. Quantisation and deployment were evaluated using accuracy, model size, latency and throughput, with laptop and on-device measurements reported separately. The resulting system is functional as a proof of concept, although its 0.11 FPS processing rate is not suitable for real-time use.

5.2 Project management

The initial timeline (May to September 2026, single-view work through July, hardware work in August and September) did not expect robustness to become two full stages rather than one small check. Both were added after the qualitative domain-shift test in Section 4.1. In hindsight this was the right call, since the two robustness findings are arguably the more original contributions of the project, but it compressed the hardware phase and led directly to running the CNN on the ARM core instead of building a custom accelerator, leaving the XNNPACK question open. The lesson is that a robustness finding worth investigating properly needs its own budgeted time, not a compression of an existing schedule. A second lesson is about ordering: the lighting stage, the last major piece of work, was finished last and left no room for anything to go wrong. It didn't, but that margin was never actually planned for (Figure 5.1).

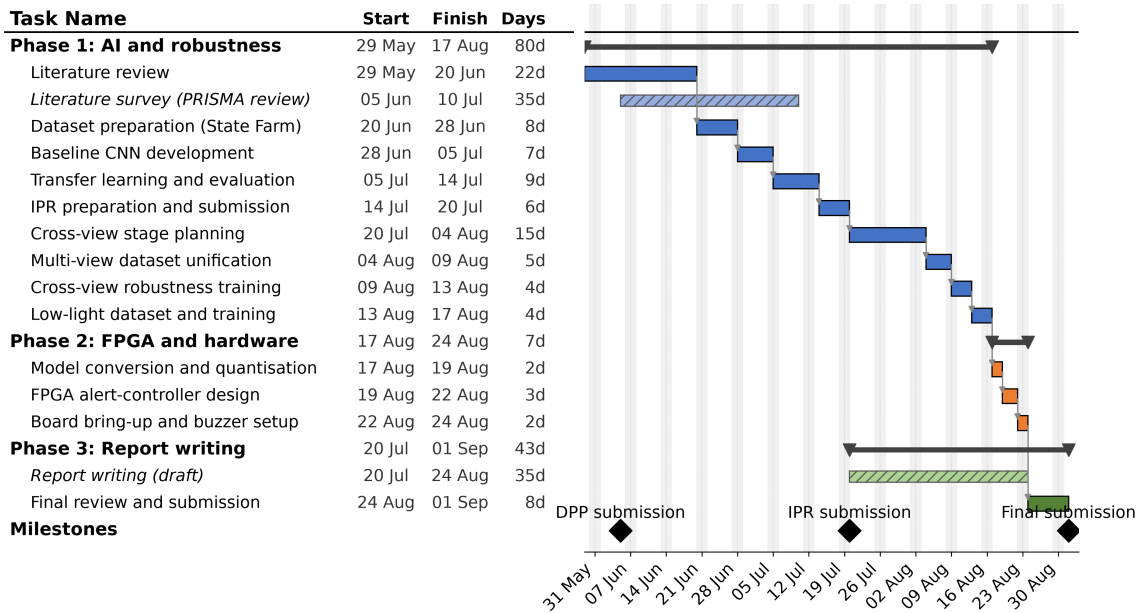


Figure 5.1: Detailed project schedule, with the literature survey and report writing shown as parallel tracks.

5.3 Insights gained

The first technical lesson was that failure on embedded hardware does not necessarily invalidate the method. The initial "Illegal instruction" crash appeared to make local inference impossible, but further investigation identified a VFPv3 and VFPv4 instruction-set mismatch. Rebuilding the runtime for the board's actual processor resolved the problem.

The second lesson was that robustness should be measured before attempting to improve it. The frozen-backbone experiments exposed substantial and uneven camera-view and lighting gaps that would have been hidden if only the fine-tuned results had been reported.

Finally, several apparent modelling or hardware failures were caused by the data pipeline. The preprocessing mismatch and the night-driver split error could both have produced believable but incorrect results. This showed the importance of checking intermediate data and predictions rather than relying only on final accuracy.

5.4 Reflection on challenges

Two unresolved challenges remain. First, the WiFi adapter intended to provide independent connectivity could not be made to work. A USB power limitation was tested by removing competing devices and freeing approximately 1 A, but the same register-access timeouts remained. This ruled against the original power explanation and pointed instead to a driver or chipset incompatibility. The issue was deprioritised because local inference itself did not require a network connection.

Second, inference remained slow without XNNPACK acceleration. Re-enabling the accelerated path might improve performance, but it could also reintroduce processor-instruction problems. Testing it required a lengthy rebuild and risked damaging the stable environment needed for the demonstration. Retaining the verified but slower configuration was therefore a deliberate

risk-management decision. The resulting 9.15-second latency is reported openly and remains the main technical limitation.

5.5 Comparison to the literature

Chapter 2 noted that the cross-view literature mostly reports post-fix accuracy with no equivalent unfixed baseline. This project’s frozen-to-fine-tuned result (75.94% to 91.76%) points the same way as Bhalla et al. (2025) and Sengar et al. (2023), even though both use more advanced methods. What this project adds is the size of the untreated gap on a standard architecture, not reported in either study and needed to judge how much of a published improvement comes from the method rather than just from having multi-view data at all.

On lighting, the 19.53% collapse is more extreme than the gradual drop Wang and Wu (2023) report across different recording conditions and gives a real measured reference point for the concern behind Shen and Lin (2023)’s image-translation work. Their synthetic approach is justified by how severe this problem is and the fact that real night data recovers 51.6 points suggests real data is still the stronger fix where it can be obtained. On hardware, Lu et al. (2025)’s 13.05 FPS for a dedicated accelerator on the same XC7Z020 chip puts this project’s 0.11 FPS in perspective - a gap of roughly two orders of magnitude, the direct cost of the decision not to build one. This project’s hardware contribution was never going to be a faster result; it is a numerically verified one instead.

5.6 Limitations

Test partitions are small in terms of subjects. Four held-out State Farm drivers and two low-light night drivers are still a small group, however many images they produce, since images from the same driver are not independent samples of anything.

Label quality in two datasets is not perfect and this is documented rather than hidden. SAM-DD’s class mapping was rebuilt by visually checking images, with a confidence level recorded per row and four low-light classes still carry an unresolved hand-side problem. No demographic information exists in any dataset used here, so no fairness claim can be made either way and SAM-DD is recorded on a simulator, not in a moving vehicle. The FPGA results are post-synthesis estimates covering the alert stage only. Two further datasets, AUC V2 and 100-Driver, were requested but not granted within the project timeline.

5.7 Commercial context and future work

The commercial case from Chapter 1 still holds, with one big qualification. Component costs stay realistic for a retrofit or fleet product: the alert logic takes up under 0.1% of a low-cost FPGA, the quantised model is 2.6 MB and the classifier runs on ordinary ARM hardware. But 0.11 FPS is not a workable product figure - at roughly nine seconds per inference and eight ticks needed before an alert fires, this setup would take over a minute to respond to sustained distraction and that has to close before any commercial claim is believable. The non-technical risks are, if anything, bigger: driver monitoring reads as surveillance to a lot of people and false alerts create liability risk while training users to ignore the system.

Four directions follow from this. First, inference speed: turning the runtime’s accelerated code path back on is a clear next step and beyond that, Table 4.6 makes the case for a dedicated CNN accelerator, since the pipeline’s whole cost sits in the classifier. Second, stronger cross-domain methods, using the contrastive viewpoint-invariant approach from Bhalla et al. (2025) on the unified dataset built here, with the same idea applying to the lighting domain too. Third, broadening the data: the AUC V2 and 100-Driver requests would add two more independent camera sources and the 45,600 daytime infrared images left out of the lighting experiment would allow a three-way comparison - day-RGB, day-infrared and night-infrared - separating the lighting gap from the sensor-modality gap. Fourth, repeated-seed runs, to supply the variance data this report has to report as missing.

5.8 Conclusion

This project examined three connected claims that performance under one camera view does not guarantee equivalent performance under another, that lighting creates an additional and potentially larger domain shift and that a detection-and-alert pipeline can be demonstrated on low-cost embedded hardware. The findings support these claims within the datasets, samples and hardware configurations evaluated in this project.

MobileNetV2 achieved 84.67% accuracy under subject-wise evaluation on the State Farm side-view dataset. In the cross-view experiment, the frozen-backbone model reached 75.94%, with uneven performance between front and side views, while fine-tuning on real multi-view data increased accuracy to 91.76%. The same daytime model achieved only 19.53% on real infrared-illuminated night footage. Training with real night data recovered performance to 71.13%, although daytime accuracy fell by 3.5 percentage points.

INT8 quantisation reduced the model from 20.78 MB to 2.60 MB at a cost of 2.7 accuracy points on the deployment sample. The model was successfully run on the ZC702 and reproduced the reference INT8 predictions, but its approximately 9.15-second inference time was around 150× slower than the laptop figure. The alert-decision stage was separately verified using real model outputs and required only 18 LUTs and 13 flip-flops in post-synthesis results.

Taken together, the results support a simple methodological lesson: robustness should be measured before it is improved, using real data where possible and the actual deployment platform when hardware performance is claimed. These choices produced less favourable numbers than controlled or estimated alternatives, but they also produced more defensible conclusions. For a system intended to operate inside a moving vehicle, that distinction matters.

References

- Abouelnaga, Y., Eraqi, H. M. and Moustafa, M. N. (2018), ‘Real-time distracted driver posture classification’. arXiv:1706.09498 [cs.CV].
- Abushahla, H. A., Varam, D., Panopio, A. J. N. and AlHajri, M. I. (2025), ‘Neural network quantization for microcontrollers: A comprehensive survey of methods, platforms, and applications’. arXiv:2508.15008 [cs.LG].
URL: <https://arxiv.org/abs/2508.15008>
- Aljasim, M. and Kashef, R. (2022), ‘E2DR: A deep learning ensemble-based driver distraction detection with recommendations model’, *Sensors* **22**(5), 1858.
- AlShalfan, K. A. and Zakariah, M. (2021), ‘Detecting driver distraction using deep-learning approach’, *Computers, Materials & Continua* **68**(1), 689–704.
- AMD/Xilinx (2026), ‘PYNQ: Python productivity for Zynq’.
URL: <http://www.pynq.io/>
- BCS, The Chartered Institute for IT (2026), ‘BCS code of conduct’.
URL: <https://www.bcs.org/membership-and-registrations/become-a-member/bcs-code-of-conduct/>
- Bhalla, A., Hellert, C. and Kasneci, E. (2025), ‘Cross-view cross-modal unsupervised domain adaptation for driver monitoring system’. arXiv:2511.12196 [cs.CV].
URL: <https://arxiv.org/abs/2511.12196>
- Dontoh, A., Ivey, S., Sirbaugh, L., Danyo, A. and Aboah, A. (2025), ‘Visual dominance and emerging multimodal approaches in distracted driving detection: A review of machine learning techniques’. arXiv:2505.01973 [cs.CV].
URL: <https://arxiv.org/abs/2505.01973>
- Eraqi, H. M., Abouelnaga, Y., Saad, M. H. and Moustafa, M. N. (2019), ‘Driver distraction identification with an ensemble of convolutional neural networks’, *Journal of Advanced Transportation* **2019**, 4125865.
- Google (2026), ‘TensorFlow lite: ML for mobile and edge devices’.
URL: <https://www.tensorflow.org/lite>
- Hariharan, J., Rama Varior, R. and Karunakaran, S. (2023), ‘Real-time driver monitoring systems on edge AI device’. arXiv:2304.01555 [cs.CV].
URL: <https://arxiv.org/abs/2304.01555>

- Horgan, J., Hughes, C., McDonald, J. and Yogamani, S. (2021), ‘Vision-based driver assistance systems: Survey, taxonomy and advances’. arXiv:2104.12583 [cs.CV].
URL: <https://arxiv.org/abs/2104.12583>
- Hussien, M., Khalil, M. and Abbas, H. (2020), ‘Novel driver distractions dataset with low lighting support’. Version 2.
URL: <https://data.mendeley.com/datasets/ykmr99nrsg/2>
- Janveja, I., Subramanian, S., Masrur, A. and Rahman, T. (2020), InSight: Monitoring the state of the driver in low-light conditions, *in* ‘Proceedings of the 18th Conference on Embedded Networked Sensor Systems’, pp. 1–14.
- Jegham, I., Ben Khalifa, A., Alouani, I. and Mahjoub, M. A. (2019), MDAD: A multimodal and multiview in-vehicle driver action dataset, *in* ‘18th International Conference on Computer Analysis of Images and Patterns (CAIP)’, Salerno, Italy.
- Kashevnik, A., Shchedrin, R., Kaiser, C. and Stocker, A. (2021), ‘Driver distraction detection methods: A literature review and framework’, *IEEE Access* **9**, 60063–60076.
- Lu, A., Fan, X., Tian, J. and Yang, X. (2025), A hardware-software co-design FPGA acceleration platform for driver behavior recognition based on MobileNetV2, *in* ‘2025 IEEE 2nd International Conference on Deep Learning and Computer Vision (DLCV)’.
- Ortega, J. D., Kose, N., Cañas, P., Chao, M.-A., Unnervik, A., Nieto, M., Otaegui, O. and Salgado, L. (2020), ‘DMD: A large-scale multi-modal driver monitoring dataset for attention and alertness analysis’. arXiv:2008.12085 [cs.CV].
URL: <https://arxiv.org/abs/2008.12085>
- Page, M. J., McKenzie, J. E., Bossuyt, P. M., Boutron, I., Hoffmann, T. C., Mulrow, C. D., Shamseer, L., Tetzlaff, J. M., Akl, E. A., Brennan, S. E., Chou, R., Glanville, J., Grimshaw, J. M., Hróbjartsson, A., Lalu, M. M., Li, T., Loder, E. W., Mayo-Wilson, E., McDonald, S., McGuinness, L. A., Stewart, L. A., Thomas, J., Tricco, A. C., Welch, V. A., Whiting, P. and Moher, D. (2021), ‘The PRISMA 2020 statement: An updated guideline for reporting systematic reviews’, *BMJ* **372**, n71.
- Parra, D., Escobar Sanabria, D. and Camargo, C. (2023), ‘A methodology and open-source tools to implement convolutional neural networks quantized with TensorFlow lite on FPGAs’, *Electronics* **12**(20), 4367.
- PRISMA (2020), ‘PRISMA 2020 flow diagram’.
URL: <https://www.prisma-statement.org/prisma-2020-flow-diagram>
- Saad, M. H., Khalil, M. I. and Abbas, H. M. (2020), End-to-end driver distraction recognition using novel low lighting support dataset, *in* ‘2020 15th International Conference on Computer Engineering and Systems (ICCES)’, pp. 1–6.
- Sandler, M., Howard, A., Zhu, M., Zhmoginov, A. and Chen, L.-C. (2018), MobileNetV2: Inverted residuals and linear bottlenecks, *in* ‘Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition’, pp. 4510–4520.

- Sengar, N., Kumari, I., Lee, J. and Har, D. (2023), ‘PoseViNet: Distracted driver action recognition framework using multi-view pose estimation and vision transformer’, *IEEE Sensors Journal* . arXiv:2312.14577.
URL: <https://arxiv.org/abs/2312.14577>
- Shen, C.-H. and Lin, S.-H. (2023), Driver distraction detection for daytime and nighttime with unpaired visible and infrared image translation, in ‘2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)’.
- State Farm (2016), ‘State farm distracted driver detection’. Kaggle competition dataset.
URL: <https://www.kaggle.com/competitions/state-farm-distracted-driver-detection>
- Tan, M. and Le, Q. V. (2019), EfficientNet: Rethinking model scaling for convolutional neural networks, in ‘Proceedings of the 36th International Conference on Machine Learning’, Vol. 97 of *Proceedings of Machine Learning Research*, pp. 6105–6114.
- Valeriano, L. C., Napoletano, P. and Schettini, R. (2018), Recognition of driver distractions using deep learning, in ‘2018 IEEE 8th International Conference on Consumer Electronics – Berlin (ICCE-Berlin)’.
- Wang, J. and Wu, Z. (2023), ‘Driver distraction detection via multi-scale domain adaptation network’, *IET Intelligent Transport Systems* .
- Wang, Y. et al. (2023), ‘100-driver: A large-scale, diverse dataset for distracted driver classification’.
URL: <https://zenodo.org/records/8335433>
- Xilinx (2019), ‘ZC702 evaluation board for the Zynq-7000 XC7Z020 SoC user guide (UG850 v1.7)’.
URL: <https://www.mouser.com/datasheet/2/903/ug850-zc702-eval-bd-1596187.pdf>
- Yang, H., Liu, H., Hu, Z., Nguyen, A.-T., Guerra, T.-M. and Lv, C. (2024), ‘Quantitative identification of driver distraction: A weakly supervised contrastive learning approach’, *IEEE Transactions on Intelligent Transportation Systems* **25**(2), 2034–2045.

Appendix A

Systematic Search and Screening Detail

Supporting evidence for Section 2.1. Literature search completed in June 2026. Table A.1 shows the per-source distribution of the 90 records found before duplicates were removed.

Table A.1: Search sources and initial records identified (before duplicate removal).

Source	Records	Main focus of results
IEEE Xplore	32	Driver distraction, CNNs, FPGA, low-light NoIR and IR/NIR studies
Google Scholar	24	Citation search, arXiv papers, dataset papers, review studies
SpringerLink	8	Driver monitoring, edge/cloud systems, deep learning
Elsevier / ScienceDirect-related	8	Intelligent systems, driver behaviour, accident-related monitoring
MDPI	7	Sensors, driver state monitoring, multimodal recognition
Dataset repositories, citation tracking	6	Mendeley low-light dataset, DMD, Zenodo dataset records
arXiv and preprint sources	5	DMD, edge AI, synthetic data, emerging methods
Total identified	90	Records before duplicate removal

Exclusion criteria at the title and abstract screening stage include scope misalignment with camera-based driver monitoring, excessive generalization with no technical specifics, or technical incompleteness. Exclusion criteria at the full-text assessment stage include lack of dataset, methodology, evaluation result, or technical contribution. Inclusion criteria cover studies with RGB, low-light, NoIR, IR, NIR, depth, thermal, or multi-modal sensing, and studies providing dataset, camera setup, evaluation metric, or data split protocol.

Appendix B

Dataset Splits and Composition

Supporting evidence for Section 3.2.4 and Table 3.1 is provided. Table B.1 lists State Farm driver identifiers for each partition. These identifiers were set at the very beginning and used without changes in all baseline, cross-view, and lighting stages making sure that there was no driver migration between partitions throughout the whole project.

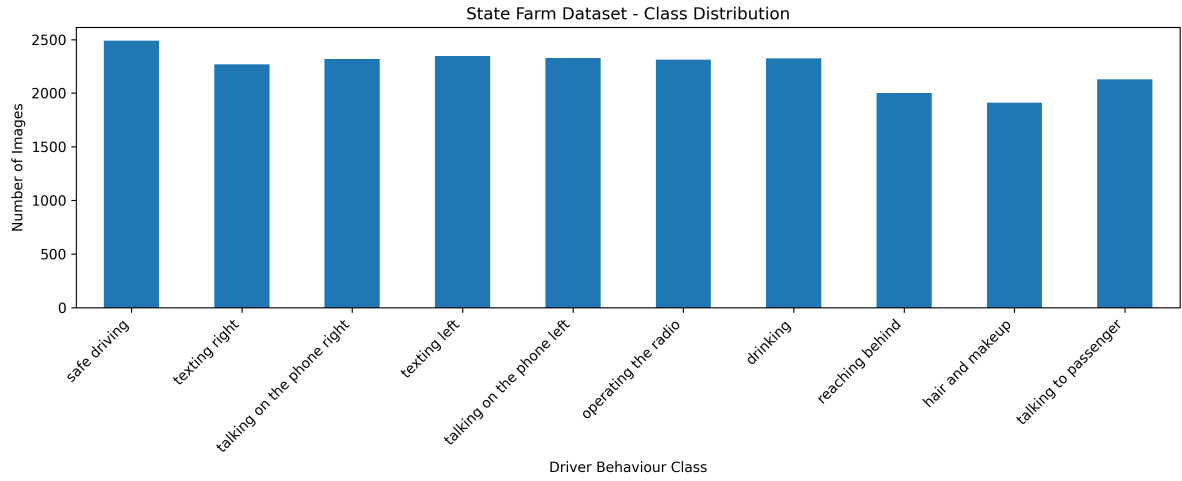
Table B.1: State Farm subject-wise split, by driver identifier.

Split	Images	Drivers	Driver identifiers
Train	15,899	18	p014, p015, p016, p021, p022, p024, p039, p042, p047, p049, p051, p052, p056, p061, p064, p066, p072, p081
Validation	3,439	4	p026, p035, p041, p050
Test	3,086	4	p002, p012, p045, p075

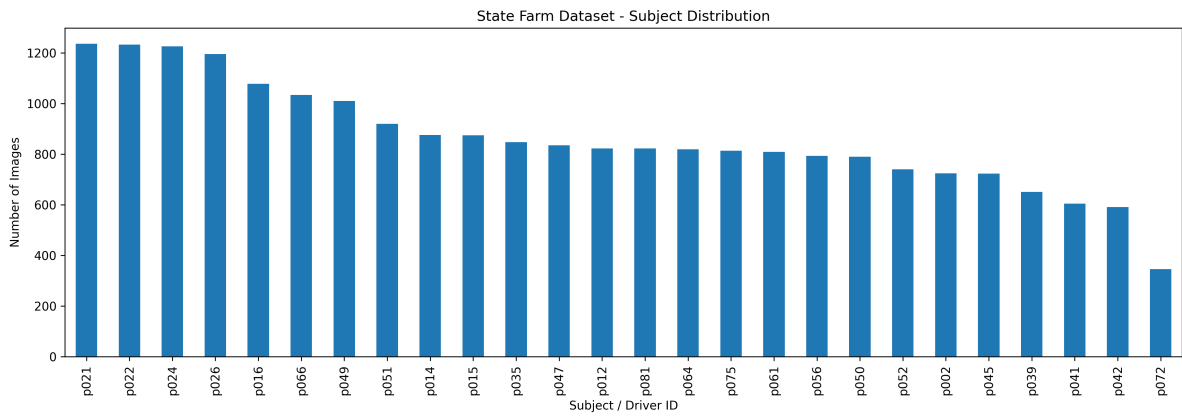
Table B.2: Composition of the unified multi-view metadata table (170,316 rows).

Dataset	View	Lighting	Train	Val	Test
State Farm	side	day	15,899	3,439	3,086
SAM-DD	front	indoor artificial	32,156	8,162	7,453
SAM-DD	side	indoor artificial	32,156	8,162	7,453
Low-Light DD	side	night	4,500	750	1,500
Low-Light DD	side	day	31,425	6,675	7,500
Cross-view experiment (State Farm + SAM-DD)			80,211	19,763	17,992
Lighting experiment (State Farm + Low-Light night)			20,399	4,189	4,586

The rows of Low-Light DD daytime split are included into consolidated table but excluded from lighting experiment intentionally, in line with the reasoning provided in Section 3.2.3. Figures B.1a and B.1b show the distribution of State Farm classes and number of images per driver respectively.



(a) Class distribution



(b) Images per driver

Figure B.1: State Farm dataset composition.

Appendix C

Full Per-Class Classification Reports

Supporting evidence for Sections 4.1, 4.2, 4.3, and 4.7 is provided. All figures are calculated based on the test partitions of held-out subjects from Appendix B.

Table C.1: Per-class F1 for the three single-view baseline models, State Farm test set.

Class	Baseline CNN	MobileNetV2	EfficientNet-B0	Support
c0 safe driving	0.620	0.707	0.738	316
c1 texting right	0.747	0.788	0.888	325
c2 phone right	0.928	0.970	0.966	338
c3 texting left	0.901	0.974	0.971	322
c4 phone left	0.944	0.917	0.880	345
c5 operating radio	0.783	0.960	0.900	327
c6 drinking	0.884	0.872	0.817	311
c7 reaching behind	0.698	0.935	0.878	293
c8 hair and makeup	0.619	0.708	0.600	251
c9 talking to passenger	0.403	0.572	0.520	258
Macro average	0.753	0.840	0.816	3,086
Weighted average	0.766	0.849	0.828	3,086

Table C.2: MobileNetV2 full per-class report, State Farm subject-wise test set (accuracy 84.67%).

Class	Precision	Recall	F1	Support
c0 safe driving	0.664	0.756	0.707	316
c1 texting right	0.960	0.668	0.788	325
c2 phone right	0.985	0.956	0.970	338
c3 texting left	0.972	0.975	0.974	322
c4 phone left	0.926	0.907	0.917	345
c5 operating radio	1.000	0.924	0.960	327
c6 drinking	0.819	0.932	0.872	311
c7 reaching behind	0.971	0.901	0.935	293
c8 hair and makeup	0.631	0.805	0.708	251
c9 talking to passenger	0.567	0.578	0.572	258
Macro average	0.849	0.840	0.840	3,086
Weighted average	0.861	0.847	0.849	3,086

Table C.3: Cross-view MobileNetV2, frozen-backbone against fine-tuned, mixed-view test set (17,992 images).

Class	Support	Frozen-backbone (75.94%)			Fine-tuned (91.76%)		
		P	R	F1	P	R	F1
safe_driving	9,006	0.887	0.881	0.884	0.944	0.977	0.960
texting_right	1,309	0.687	0.502	0.580	0.924	0.765	0.837
phone_right	1,332	0.722	0.642	0.680	0.840	0.986	0.907
texting_left	1,286	0.422	0.768	0.545	0.974	0.821	0.891
phone_left	1,341	0.591	0.782	0.673	0.906	0.899	0.903
adjusting_radio	327	0.928	0.746	0.827	0.974	0.927	0.950
drinking	1,273	0.754	0.610	0.675	0.840	0.833	0.837
reaching_behind	835	0.884	0.695	0.778	0.953	0.884	0.917
hair_or_makeup	1,025	0.696	0.537	0.606	0.867	0.876	0.871
talking_to_passenger	258	0.614	0.136	0.222	0.604	0.519	0.558
Macro average	17,992	0.719	0.630	0.647	0.883	0.849	0.863
Weighted average	17,992	0.781	0.759	0.761	0.918	0.918	0.916

Table C.4: Lighting stage: day-only baseline (night-only test set, 1,500 images), frozen-backbone and fine-tuned (merged day+night test set, 4,586 images). The baseline’s zero-recall rows are the collapse described in Section 4.3.

Class	Baseline (19.53%)		Frozen (62.30%)			Fine-tuned (77.91%)		
	P	R	P	R	F1	P	R	F1
safe_driving	0.000	0.000	0.559	0.712	0.626	0.551	0.845	0.667
texting_right	0.000	0.000	0.721	0.451	0.554	0.986	0.459	0.626
phone_right	0.000	0.000	0.689	0.598	0.640	0.900	0.883	0.891
texting_left	0.170	0.993	0.720	0.676	0.697	0.965	0.648	0.776
phone_left	0.000	0.000	0.780	0.523	0.626	0.926	0.653	0.765
adjusting_radio	0.000	0.000	0.732	0.851	0.787	0.998	0.960	0.979
drinking	0.000	0.000	0.392	0.764	0.518	0.638	0.889	0.743
reaching_behind	0.233	0.960	0.784	0.858	0.819	0.880	0.975	0.925
hair_or_makeup	0.000	0.000	0.423	0.414	0.419	0.596	0.663	0.628
talking_to_passenger	0.000	0.000	0.820	0.336	0.477	0.719	0.821	0.767
Macro average	0.040	0.195	0.662	0.618	0.616	0.816	0.780	0.777
Weighted average	0.040	0.195	0.664	0.623	0.621	0.822	0.779	0.779

Appendix D

Quantisation Raw Figures

Supporting evidence for Section 4.4 and Table 4.6 is provided. Exact file sizes have been read from the filesystem. Megabytes are calculated in bytes divided by 1024^2 as in project’s results tables.

Table D.1: Exact model artefact sizes.

Artefact	Format	Bytes	MB
best_mobilenetv2_crossview_finetuned.keras	Keras float32	21,791,872	20.78
mobilenetv2_crossview_finetuned_int8.tflite	TFLite INT8	2,723,448	2.60
mobilenetv2_crossview_int8.tflite	TFLite INT8	2,721,672	2.60
mobilenetv2_crossview_dynamicrange.tflite	TFLite dynamic range	2,518,480	2.40
Compression ratio, float32 to INT8			8.00×

Table D.2: Quantisation before/after measurements. All samples are drawn from held-out subject-wise test partitions.

Model	Sample	n	Float32	INT8	Agreement
Cross-view frozen	mixed front+side	500	76.80%	76.00%	91.2%
Cross-view frozen	random side	300	59.30%	54.70%	—
Cross-view fine-tuned	random side	300	80.00%	77.30%	—
Cross-view fine-tuned	held-out sample	100	Keras vs. TFLite		100/100
Cross-view fine-tuned	held-out sample	100	board vs. laptop		100/100

The two rows of board-versus-laptop agreement illustrate the measurements proving the numerical correctness of from-source ARM32 runtime (Section 4.5.1). Dynamic-range variant was created and validated but not deployed since the INT8 variant was used in the accuracy evaluation.

Appendix E

FPGA Synthesis Report Excerpts

Supporting evidence for Section 4.6 is provided. The excerpts are taken verbatim from Vivado program created by Vivado 2017.4 targeting "7a35tcpg236-1" with the design status marked as “Synthesized” and provided out of context.

Table E.1: Slice logic utilisation (from `utilization.rpt`).

Site type	Used	Available	Util.%
Slice LUTs	18	20,800	0.09
LUT as logic	18	20,800	0.09
LUT as memory	0	9,600	0.00
Slice registers	13	41,600	0.03
Register as flip-flop	13	41,600	0.03
Register as latch	0	41,600	0.00
F7 muxes	0	16,300	0.00
F8 muxes	0	8,150	0.00
Block RAM	0	—	0.00
DSP slices	0	—	0.00

Utilization

Timing

Q

≡

≡

>> Hierarchy

Name	Slice LUTs (20800)	Slice Registers (41600)	Bonded IOB (106)	BUFGCTRL (32)
distraction_alert_controller	18	13	19	1

Figure E.1: Vivado’s own utilization report view for the synthesized alert controller, confirming the 18 LUTs / 13 registers reported in Table E.1.

All 13 registers are clock-enabled flip-flops with synchronous reset. Vivado’s report notes that “the final LUT count, after physical optimizations and full implementation, is typically lower,” hence the claim in Section 4.6 about 18 LUTs being conservative post-synthesis estimate rather than post-placement-and-routing.

Table E.2: Design timing summary, clock constrained at 100 MHz.

Metric	Value	Failing endpoints
Worst negative slack (WNS)	+7.276 ns	0 of 17
Total negative slack (TNS)	0.000 ns	—
Worst hold slack (WHS)	+0.272 ns	0 of 17
Total hold slack (THS)	0.000 ns	—
Worst pulse width slack (WPWS)	+4.500 ns	0 of 13

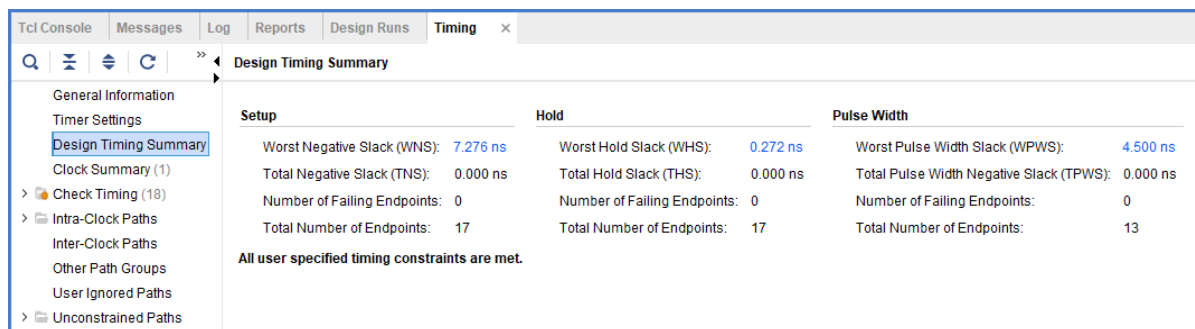


Figure E.2: Vivado's timing summary panel, confirming the worst-negative-slack figure reported in Table E.2.

Appendix F

Alert Delivery Path and Its Verification

Evidence in support of the alert-delivery pipeline is summarized in Chapter 3. The board runs an endless capture-classify loop that uses hysteresis logic as a straight copy of "distraction_alert_controller.v" in Python. It uses the same 40% confidence threshold, the same eight-tick set and clear counters, and follows the exact same process of incrementing the counters and checking set and clear ticks in relation to the increments; hence, a "low confidence" tick leaves both counters unaffected in the same way as the RTL always-block does. On the rising edge of an alert, and on that alone (and not once per classified frame), the system posts the class, confidence, and the JPEG-encoded triggering frame to the FastAPI backend. The backend saves the image and serves it to the Kotlin Android client via the same bearer token authentication as the rest of the API.

An end-to-end trigger would require keeping a distracted position in front of the camera for about seventy seconds due to the inference speed of the board. Consequently, validation was done in three independent layers.

Table F.1: Three-layer verification of the alert-delivery path.

Layer	What was exercised	Result
1. Logic equivalence	The Python hysteresis port replayed against the same five scenarios as the Verilog testbench (sustained safe, sub-threshold burst, sustained distraction, an uncertain tick mid-run, sustained safe again).	10/10 checks passed, matching the testbench's expected transitions.
2. Hardware execution	Real webcam frames through the real INT8 model and the real hysteresis state machine, on the board, with no synthetic data.	Ran cleanly end to end at ~ 9.1 s per tick, no crash and no false alert.
3. Network delivery	The production posting function imported and called directly, then the stored record retrieved from the backend and viewed in the Android client.	HTTP 200; record and image confirmed present and byte-identical.

Layer 1 shows that the deployed decision logic behaves the same as the hardware design verified in Section 4.6. Layer 2 shows that the full pipeline runs on real hardware. Layer 3 shows that the network call the loop actually makes reaches its destination. What the three layers together do not add up to is one single unattended physical demonstration. That limitation is stated here directly, not hidden.

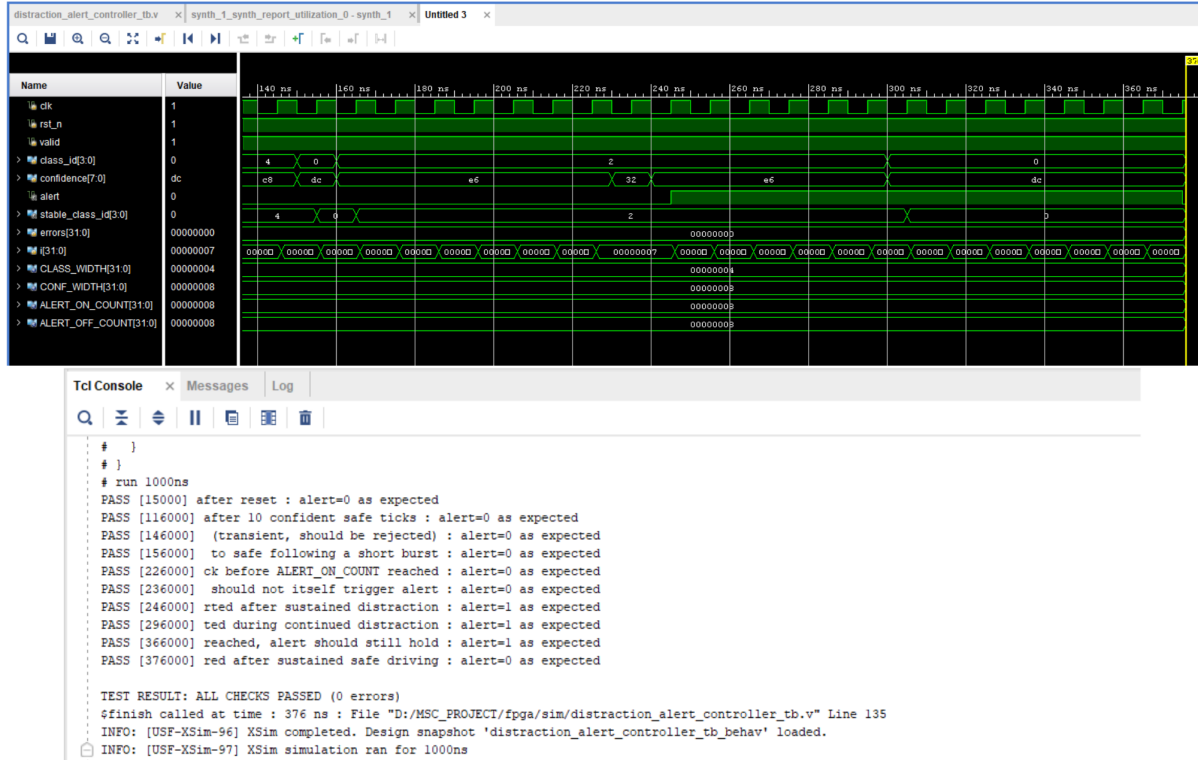


Figure F.1: The self-checking testbench run in Vivado’s simulator: the waveform shows **alert** correctly staying low, then rising after sustained distraction and clearing after sustained safe driving, and the Tcl Console confirms all ten checks passed, matching Table F.1 (Layer 1).

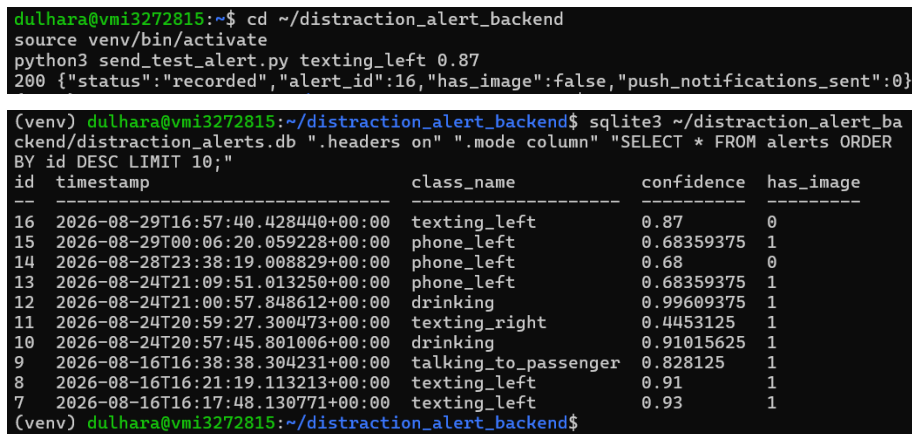


Figure F.2: The FastAPI backend receiving a real alert (top) and the same record persisted in its SQLite database (bottom), confirming the HTTP 200 and stored-record result reported in Table F.1.



Figure F.3: A real alert received on the Android client, showing the triggering frame and timestamp posted by the board.

Appendix G

Physical Buzzer Alert Output

Support of the buzzer paragraph in Chapter 3 is provided in this appendix. This appendix covers why the board could not drive a buzzer at all before this work, what was built to fix that, the real problems hit along the way, and how the buzzer itself was diagnosed and driven correctly.

G.1 The board could not control the header pin at all

The available buzzer is a real two-pin passive one, having no transistor or driver circuit at all, and therefore requires a direct connection to a GPIO header pin. Inspecting `/sys/class/gpio` of the board's stock PYNQ image, only a single GPIO controller is found there, `"zynq_gpio"`, with none of those going to the board's PMOD headers. The PMOD and LED-related pins are on the FPGA-fabric I/O banks, not the processor's fixed MIO banks. Furthermore, there is no bitstream or overlay exposing those pins anywhere on the board's filesystem. Therefore, no part on the board can toggle a header pin from Python without a bitstream doing this job explicitly. This was determined directly and not through assumptions.

G.2 The overlay

A minimal Vivado project was created from scratch targeting the real device on the board. Project's Basys3 target used in Section 3.5). The block design (Figure G.1) includes a Zynq7 Processing System block and a single-bit AXI GPIO block. The processing-system block does not reconfigure. Linux is already running from the SD card independently of this design. It is included so Vivado generates the AXI clock, reset and address decode connections allowing the running processor to address a peripheral that exists in the reconfigurable fabric. The output of the GPIO block was connected to package pin V7, marked by the board's user guide as pin 1 of header J62 named as "PMOD2". (Xilinx, 2019).

Three concrete build errors were faced and solved via reading Vivado's output messages rather than guessing. A manual clock connection automation command failed because the connection of the clock via AXI4 connection automation had already been done. After synthesis, implementation, and bitstream generation had finished without errors.

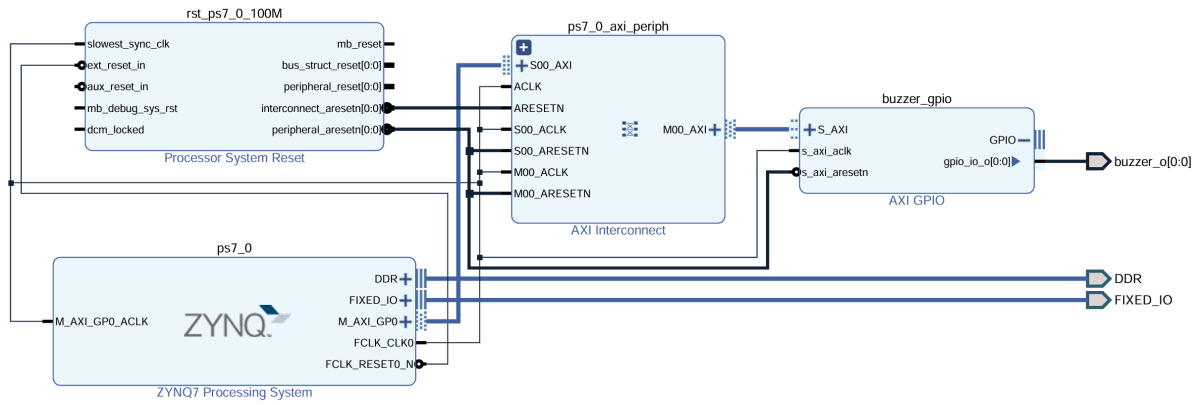


Figure G.1: The buzzer overlay’s Vivado block design. A Zynq7 Processing System block provides AXI/clock/reset plumbing to the already-running processor. A single-bit AXI GPIO block is the only new circuit, exposed externally as **buzzer_o** on package pin V7.

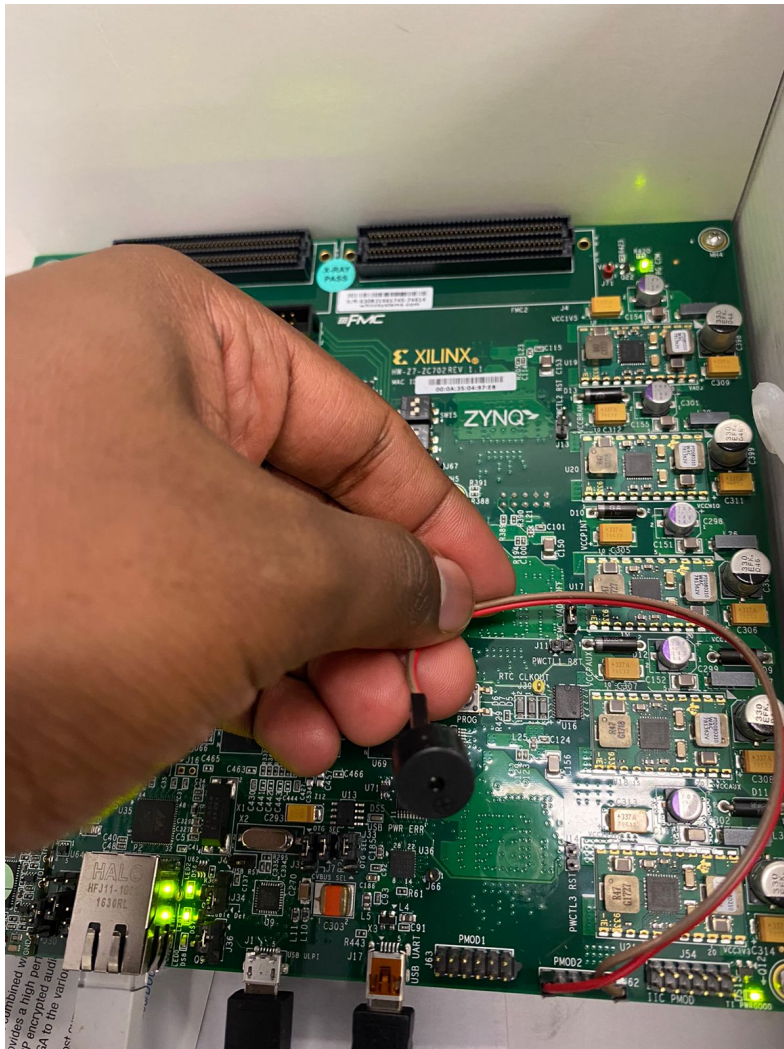


Figure G.2: The ZC702 board with the passive buzzer wired directly to header pin J62 (PMD02_0), driven by the GPIO overlay described in this appendix.

G.3 Root permission and two unrelated Jupyter defects

The loading of the hardware overlay failed because of a permission issue. Even though initially it seemed like a file permission issue, this was not the case. The software has an intrinsic requirement to be run with root privileges and will not work with another type of privileges regardless of file permissions. Initial attempts to solve the problem through file permissions modification were investigated and eventually proved ineffective because the check was about execution, not file access. The right solution was to configure the notebook service to start up with root privileges, which fixed the problem right away. Such configuration corresponds to the standard, expected one for boards of this kind; the specific board being tested had a different one. In the process of troubleshooting, two other minor problems were found and fixed. One setting secretly told the notebook service to use a wrong user-specific directory which made accessing a certain package impossible later when running tests on the device and the directory that the notebook browser displayed was different from the real one containing the notebooks, making them invisible.

G.4 Driving a passive buzzer

The first attempts to power the buzzer and keep it active failed with producing only a single click rather than a constant sound. This behavior is typical for such a simple buzzer that does not have an internal circuit to keep the sound going and emits it only while switching on and off and stays silent otherwise. Therefore, the solution was to rapidly switch the buzzer on and off in software using sufficient frequency to create a sound rather than clicks. Using a precisely controlled and constantly evaluated timer, rather than a plain built-in delay, led to the sound sounding more steady but at the price of taking up a full CPU core during the alert.

G.5 Integration

The hardware interface is created once, upon program initialization, rather than being reinitialized each time an alert is sent, since this would lead to considerable delay. The buzzer and the phone notification are triggered at the very same point in code, guaranteeing their synchronous triggering. This was checked by testing in the conditions of the actual system rather than using an easy shortcut test, since it relies on having administrator privileges which were ensured by fixing the problem described above.

Appendix H

Selected Code Excerpts

Supporting evidence for Sections 3.2.1, 3.4.1 and 3.5. The three excerpts below are the exact functions responsible for the subject-wise split, the quantisation-conversion fix and the FPGA decision logic. Full files are in project's GitHub repository

Subject-wise splitting

```
1 from sklearn.model_selection import GroupShuffleSplit
2
3 SEED = 42
4
5 # First split: 70% train, 30% temporary set
6 gss1 = GroupShuffleSplit(n_splits=1, train_size=0.70, random_state=SEED)
7 train_idx, temp_idx = next(gss1.split(df, groups=df["subject"]))
8
9 train_df = df.iloc[train_idx].reset_index(drop=True)
10 temp_df = df.iloc[temp_idx].reset_index(drop=True)
11
12 # Second split: temporary set into validation and test
13 gss2 = GroupShuffleSplit(n_splits=1, train_size=0.50, random_state=SEED)
14 val_idx, test_idx = next(gss2.split(temp_df, groups=temp_df["subject"]))
```

Listing H.1: Subject-wise train/validation/test split.

The augmentation-layer conversion fix

```
1 aug_layer = None
2 for layer in model.layers:
3     if "augmentation" in layer.name.lower():
4         aug_layer = layer
5         break
6 if aug_layer is None:
7     raise ValueError("Could not find augmentation layer.")
8
9 aug_layer.call = lambda inputs, training=None: inputs
10 print("Monkey-patched augmentation layer:", aug_layer.name)
```

Listing H.2: Disabling the augmentation step so it isn't included when converting the model.

FPGA alert-controller RTL

```
1  'timescale 1ns / 1ps
2
3  module distraction_alert_controller #(
4      parameter integer CLASS_WIDTH = 4,
5      parameter integer CONF_WIDTH = 8,
6      parameter integer CNT_WIDTH = 4,
7      parameter [CLASS_WIDTH-1:0] SAFE_CLASS = 4'd0,
8      parameter [CONF_WIDTH-1:0] CONF_THRESHOLD = 8'd102,
9      parameter integer ALERT_ON_COUNT = 8,
10     parameter integer ALERT_OFF_COUNT = 8
11 );
12     input wire clk,
13     input wire rst_n,
14     input wire valid,
15     input wire [CLASS_WIDTH-1:0] class_id,
16     input wire [CONF_WIDTH-1:0] confidence,
17     output reg alert,
18     output reg [CLASS_WIDTH-1:0] stable_class_id
19 );
20     reg [CNT_WIDTH-1:0] distracted_run;
21     reg [CNT_WIDTH-1:0] safe_run;
22     wire is_distracted = valid && (class_id != SAFE_CLASS) && (confidence >= CONF_THRESHOLD);
23     wire is_safe = valid && (class_id == SAFE_CLASS) && (confidence >= CONF_THRESHOLD);
24     always @(posedge clk) begin
25         if (!rst_n) begin
26             alert <= 1'b0;
27             stable_class_id <= SAFE_CLASS;
28             distracted_run <= {CNT_WIDTH{1'b0}};
29             safe_run <= {CNT_WIDTH{1'b0}};
30         end else begin
31             if (valid)
32                 stable_class_id <= class_id;
33             if (is_distracted) begin
34                 if (distracted_run < ALERT_ON_COUNT[CNT_WIDTH-1:0])
35                     distracted_run <= distracted_run + 1'b1;
36                 safe_run <= {CNT_WIDTH{1'b0}};
37             end else if (is_safe) begin
38                 if (safe_run < ALERT_OFF_COUNT[CNT_WIDTH-1:0])
39                     safe_run <= safe_run + 1'b1;
40                 distracted_run <= {CNT_WIDTH{1'b0}};
41             end
42             if (!alert && is_distracted && (distracted_run >= (ALERT_ON_COUNT[CNT_WIDTH-1:0] - 1'b1)))
43                 alert <= 1'b1;
44             else if (alert && is_safe && (safe_run >= (ALERT_OFF_COUNT[CNT_WIDTH-1:0] - 1'b1)))
45                 alert <= 1'b0;
46         end
47     end
48 endmodule
```

Listing H.3: The alert-controller logic: only raises or clears an alert after eight confident readings in a row, so a single flickered guess can't trigger it.

Appendix I

Repository and Notebook Map

The project is also published as a working repository on GitHub, where the same code, notebooks and results shown throughout this report can be browsed and downloaded directly, at <https://github.com/LangaXD/FPGA-Based-Driver-Distracton-Detection-System-Using-Image-Classification>.

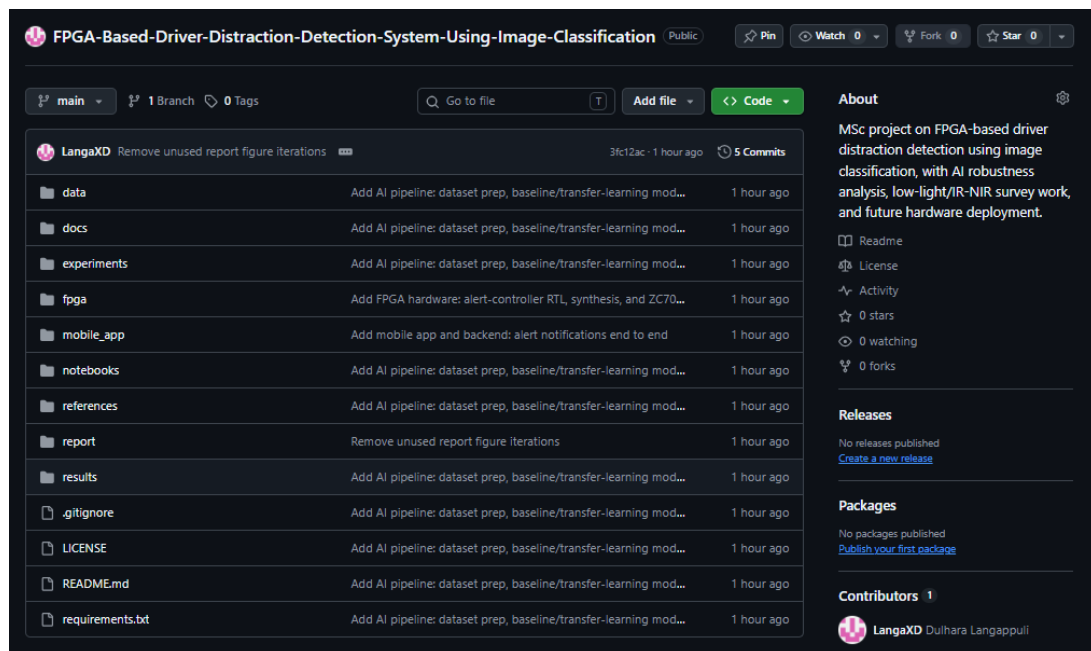


Figure I.1: The project's GitHub repository.